Optimization

Muchang Bahng

Spring 2025

Contents

Definition 0.1 (Rosenbrock Function) . . . 3

1 Linear Programming **4**
1.1 The Duality Theorem . . . 4
Lemma 1.1 (Dual Formulation of Span) . . . 4
Theorem 1.2 (Separating Hyperplane Theorem) . . . 4
Theorem 1.3 (Dual Formulation of Positive Span) . . . 5
Corollary 1.4 (Farkas' Lemma, Equality Form) . . . 8
Theorem 1.5 (Farkas' Lemma, Inequality Form) . . . 8
Definition 1.1 (Primal and Dual Feasibility) . . . 9
Example 1.1 (Manual Derivation of Dual from Primal) . . . 10
Lemma 1.6 (Dual of Dual is Primal) . . . 10
Lemma 1.7 (Weak Duality) . . . 11
Theorem 1.8 (Strong Duality) . . . 11
Corollary 1.9 (Primal-Dual Optimality Criterion) . . . 12
Example 1.2 (Production Planning and Resource Pricing) . . . 13
Example 1.3 (Primal Feasible, Dual Infeasible) . . . 14
Example 1.4 (Both Infeasible) . . . 14
1.2 Complementary Slackness and Sensitivity . . . 15
Theorem 1.10 (Complementary Slackness) . . . 15
Example 1.5 (Complementary Slackness for Furniture, No Slack) . . . 16
Theorem 1.11 (Dual Variables are Maringal/Shadow Prices) . . . 16
1.3 Geometry of Linear Programs . . . 17
Definition 1.2 (Extreme Point, Basic Feasible Solution) . . . 17
Lemma 1.12 (Extreme Points are Basic Feasible Solutions) . . . 17
Theorem 1.13 (Fundamental Theorem of Linear Programming) . . . 17
1.4 The Simplex Method . . . 18
Theorem 1.14 (Optimality Test) . . . 18
Example 1.6 (Simplex on the Furniture Problem) . . . 19
1.5 Interior-Point Methods and the Lagrangian View . . . 20

2 Smooth Optimization **21**
Definition 2.1 (ϵ-Critical Point) . . . 21
Definition 2.2 (μ-Strongly Convex) . . . 21
Theorem 2.1 (Taylor) . . . 21
Lemma 2.2 (Descent Lemma) . . . 21
Theorem 2.3 (Spiral) . . . 22
Theorem 2.4 (Quadratic Growth) . . . 22
Theorem 2.5 (Lightning) . . . 22
Definition 2.3 (Polyak-Lojasiewicz Inequality) . . . 23
Lemma 2.6 (Strong Convexity Implies PL-Inequality) . . . 23
Theorem 2.7 (Double Lightning) . . . 23
Theorem 2.8 . . . 23

3 Gradient Methods **25**
3.1 Newton-Raphson Method for Root Finding . . . 25
Theorem 3.1 (Convergence) . . . 25
3.2 Stochastic Gradient Descent . . . 26
Theorem 3.2 (Minibatch Gradient is an Unbiased Estimator) . . . 26

Example 3.1 (Linear Regression) . . . 27
Heuristic 3.1 (Choosing Batch Size) . . . 28
3.3 Learning Rates and Schedulers . . . 28
3.4 Momentum and Nesterov . . . 29
3.5 Block Coordinate Descent . . . 30

4 Subgradient Methods **31**
Definition 4.1 (Convex Function) . . . 31
Definition 4.2 (Subgradient) . . . 31
Example 4.1 . . . 32
4.1 Proximal Gradient Descent . . . 32
Definition 4.3 (Proximal Operator) . . . 32

5 Adaptive Gradient Methods **34**
5.1 Adagrad . . . 34
5.2 RMSProp and Adadelta . . . 34
5.3 Adam . . . 34

6 Second-Order Optimizers **35**
6.1 Newton's Method . . . 35
6.2 Gauss Newton Method . . . 35

7 Quasi-Newton Methods **36**
7.1 Secant Method . . . 36
7.2 Davidon-Fletcher-Powell (DFP) . . . 36
7.3 Broyden's Method . . . 37
7.4 Symmetric Rank-1 Update (SR1) . . . 37
7.5 BFGS . . . 37

8 Gradient Free Methods **42**
8.1 Simulated Annealing . . . 42
8.2 Nelder-Mead . . . 42

9 Lagrangian Optimizers for Constraints **43**
9.1 Lagrange Multipliers . . . 43
9.2 KKT Conditions . . . 43
9.3 ADMM . . . 43

10 Non-Lagrangian Optimizers for Constraints **44**
10.1 Penalty . . . 44
10.2 Projection . . . 44
10.3 Saddle Point Problem in Nonconvex Optimization . . . 45

11 Sparsity-Inducing Optimizers **46**
11.1 Clipping . . . 46

References **47**

Optimization is such an important tool that it deserves a set of notes in itself. All problems in model training essentially stems from non-ideal optimization. Knowing the strengths and weaknesses of each optimizer allows you to diagnose which ones to use.

Generally, we (non-exclusively) categorize optimization algorithms as such:

1. *Convex*? Convex optimization is pretty easy to solve and has been studied extensively. For nonconvex optimization, none of the algorithms can guarantee that we will find the global minima, and this is one of the hardest problems in statistics.[1]
2. *Constrained*? Is the parameter space constrained to a certain manifold?
3. *Order*. Do we use derivatives at all? First-order derivatives (gradient)? Second-order (Hessian)?

These algorithms try to solve the following potential problems.

1. *Convergence*. Do we converge to some point?
2. *Optimality*. Is this point close to the true global minima?
3. *Efficiency*. Can we iterate efficiently?

As a benchmark test, the following function will be used a lot.

Definition 0.1 (Rosenbrock Function)

The **Rosenbrock function** is defined

$$f(x,y) = (a-x)^2 + b(y-x^2)^2 \tag{1}$$

which has a global minimum at (a,b).

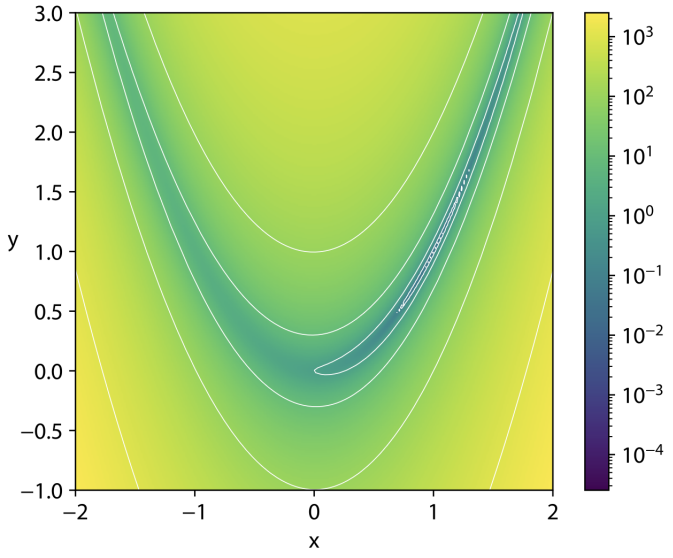


Figure 1: Typically, we set $a = 1, b = 100$.

[1] In practice, when we are doing high-dimensional nonconvex optimization, the best we can do is play around with some properties. In these cases, 0th order approximations are hopeless since the dimensions are too high, and second order approximations are hopeless either since computing the Hessian is too expensive for one run. Therefore, we must resort to some first order methods.

1 Linear Programming

1.1 The Duality Theorem

A linear program (LP) asks us to optimize a linear function over a set cut out by finitely many linear inequalities. The duality theorem lets us pair every LP (the *primal*) with a second LP (the *dual*) whose optimal value is the same, and it is the source of both optimality certificates and of the simplex method below. The route to it goes through a sequence of "theorems of the alternative" that characterize when a vector lies in the span of some generators, first with arbitrary coefficients (equalities), and then with nonnegative coefficients (inequalities).

Given vectors $v_1, \dots, v_m \in V$, we know that $v \in \text{span}\{v_i\}$ if v can be written as a linear combination of the v_i's. We wish to formulate an equivalent condition in terms of the dual space.

Lemma 1.1 (Dual Formulation of Span)

Let V be a vector space, with $v_1, \dots, v_m \in V$. For any $v \in V$, the following are equivalent.

1. *Primal.* $v \in \text{span}\{v_i\}$
2. *Dual.* For every $\ell \in V^\dagger$ satisfying $\ell(v_i) = 0$ for all i, it is the case that $\ell(v) = 0$.

Proof. We prove bidirectionally.

1. ($\to$). This is easy. Let $v \in \text{span}\{v_i\}$. Then, if $\ell(v_i) = 0$ for all i, then

$$\ell(v) = \ell\left(\sum_{i=1}^m a_i v_i\right) = \sum_{i=1}^m a_i \ell(v_i) = \sum_{i=1}^m 0 = 0 \tag{2}$$

2. ($\leftarrow$). We prove the contrapositive. Let $W = \text{span}\{v_i\}$ and $v \notin W$. Since $W \cap \text{span}\{v\} = \{0\}$, we can take their direct sum $W \oplus \text{span}\{v\}$ and define the linear functional

$$\ell : W \oplus \text{span}\{v\} \to \mathbb{F}, \quad \ell(w + av) = a \tag{3}$$

Then, $\ell \in W^\perp$ and $\ell(v) = 1$. We can extend the domain linearly—which is always possible by extending a basis of $W \oplus \text{span}\{v\}$ to a basis of V—to create a functional $\ell \in V^\dagger$.

Now, we state a theorem that allows us to check if a vector v can be written as a *nonnegative* linear combination of the v_i's. We need the separating hyperplane theorem first.

Theorem 1.2 (Separating Hyperplane Theorem)

Let $C \subset \mathbb{R}^n$ be a nonempty closed convex set, and let $v \in \mathbb{R}^n \setminus C$. Then there exists a nonzero linear functional $\ell \in (\mathbb{R}^n)^\dagger$ and $\alpha \in \mathbb{R}$ such that

$$\ell(v) < \alpha \leq \ell(c) \qquad \forall c \in C. \tag{4}$$

Equivalently, there exists a hyperplane

$$H = \{x \in \mathbb{R}^n : \ell(x) = \alpha\} \tag{5}$$

that strictly separates v from C.

Proof. Since C is nonempty and closed, there exists a point $p \in C$ minimizing the Euclidean distance to v, i.e.

$$\|v - p\| = \inf_{c \in C} \|v - c\|. \tag{6}$$

Furthermore, since $v \notin C$, we have $p \neq v$. For any $c \in C$ and $t \in [0,1]$, convexity implies

$$p + t(c-p) = (1-t)p + tc \in C. \tag{7}$$

By the minimality of p, we have

$$\|v-p\|^2 \leq \|v-p-t(c-p)\|^2. \tag{8}$$

Expanding the right-hand side gives

$$\|v-p\|^2 \leq \|v-p\|^2 - 2t\langle v-p, c-p\rangle + t^2\|c-p\|^2. \tag{9}$$

Subtracting $\|v-p\|^2$ and dividing by $t > 0$,

$$2\langle v-p, c-p\rangle \leq t\|c-p\|^2. \tag{10}$$

Taking $t \to 0^+$, we obtain

$$\langle v-p, c-p\rangle \leq 0 \qquad \forall c \in C. \tag{11}$$

Now define the linear functional

$$\ell : \mathbb{R}^n \to \mathbb{R}, \qquad \ell(x) = \langle p-v, x\rangle. \tag{12}$$

Since $p \neq v$, this functional is nonzero. Furthermore, the preceding inequality implies

$$\ell(c) \geq \ell(p) \qquad \forall c \in C. \tag{13}$$

On the other hand,

$$\begin{aligned} \ell(p) - \ell(v) &= \langle p-v, p-v\rangle && (14)\\ &= \|p-v\|^2 && (15)\\ &> 0. && (16) \end{aligned}$$

Therefore, setting $\alpha = \ell(p)$, we have

$$\ell(v) < \alpha \leq \ell(c) \qquad \forall c \in C, \tag{17}$$

as desired.

Theorem 1.3 (Dual Formulation of Positive Span)

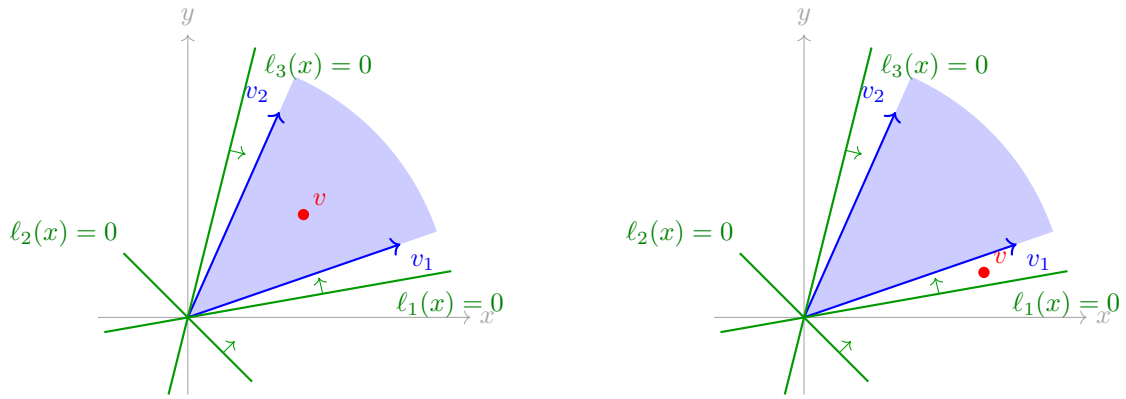
Let $V = \mathbb{R}^n$, with $v_1, \dots, v_m \in V$. For any $v \in V$, the following are equivalent.

1. *Primal.* There exists $a_i \geq 0$, $i = 1, \dots, m$ such that

$$v = \sum_{i=1}^{m} a_i v_i, \quad a_i \geq 0 \tag{18}$$

2. *Dual.* For every $\ell \in V^\dagger$ satisfying $\ell(v_i) \geq 0$ for all i, it is the case that

$$\ell(v) \geq 0 \tag{19}$$



(a) $v \in \text{cone}(v_1, v_2)$. (b) $v \notin \text{cone}(v_1, v_2)$.

Figure 2: Geometric illustration of the dual characterization of positive span. The green lines represent kernels of linear functionals, with arrows indicating their nonnegative half-planes.

Proof. We prove bidirectionally.

1. ($\to$). Let us have such an $\ell \in V^\dagger$ satisfying $\ell(v_i) \geq 0$. Then,

$$\ell(v) = \ell\left(\sum_{i=1}^m a_i v_i\right) = \sum_{i=1}^m a_i \ell(v_i) \geq 0 \tag{20}$$

since $a_i \geq 0$ and $\ell(v_i) \geq 0$.

2. ($\leftarrow$). We would like to prove the contrapositive. Let

$$C = \left\{ \sum_{i=1}^m a_i v_i \colon a_i \geq 0 \right\} \tag{21}$$

by the cone spanned by v_i. If $v \notin C$, then we have to show there exists a $\ell \in V^\dagger$ satisfying $\ell(v_i) \geq 0$ but $\ell(v) < 0$. From the visual, we basically want to construct a small hyperplane squeezed in between the cone and v such that $\ell(v) < 0$. We note three things.

   (a) C is convex since given two vectors $u = \sum_i a_i v_i$ and $w = \sum_i b_i v_i$, we can see that

$$tu + (1-t)w = \sum_i (ta_i + (1-t)b_i)v_i \in C \tag{22}$$

   (b) C is a cone, i.e. $tu \in C$ for $t \geq 0, u \in C$.

   (c) C is closed. This is the hard part. Note that a point $u \in C$ can have multiple representations $u = \sum_{i=1}^m a_i v_i$, $a_i \geq 0$. We select one minimizing $A = \sum_{i=1}^m a_i$. Such a minimizer exists since given any feasible representation with coefficient sum M, we can restrict the search to

$$S = \left\{ a \in \mathbb{R}^m_{\geq 0} \colon \sum_i a_i v_i = u, \sum_i a_i \leq M \right\} \tag{23}$$

   This is clearly bounded by $[0, M]^m$, and it is closed since we can define the continuous linear map

$$T : \mathbb{R}^m \to \mathbb{R}^n, T(a) = \sum_{i=1}^m a_i v_i \tag{24}$$

   Therefore,

$$S = T^{-1}(\{u\}) \cap \mathbb{R}^m_{\geq 0} \cap \left\{ a \in \mathbb{R}^m \colon \sum_{i=1}^m a_i \leq M \right\} \tag{25}$$

which is a finite intersection of closed sets and hence closed. By Heine-Borel, it is compact. Therefore, by the extreme value theorem, we can find such a minimizer. Now we take a convergence sequence $(u_n) \in C$ such that $u_n \to u$. We are done if we can prove $u \in C$. We do this by first choosing a minimal representation for each u_n:

$$u_n = \sum_{i=1}^{m} a_{n,i} v_i, \quad A_n = \sum_{i=1}^{m} a_{n,i} \tag{26}$$

We claim that (A_n) is bounded. Suppose it isn't. Then, by passing to a subsequence, we can assume that $A_n \to \infty$. Because $u_n \to u$, (u_n) is bounded, so

$$\frac{u_n}{A_n} \to 0 \tag{27}$$

But

$$\frac{u_n}{A_n} = \sum_{i=1}^{m} \underbrace{\frac{a_{n,i}}{A_n}}_{q_{n,i}} v_i, \quad q_{n,i} \geq 0, \sum_{i=1}^{m} q_{n,i} = 1 \tag{28}$$

Therefore, the vectors $(q_{n,1}, \dots q_{n,m})$ lie in the standard simplex

$$\Delta^{m-1} = \left\{ q \in \mathbb{R}^m \colon q_i \geq 0, \sum_i q_i = 1 \right\} \tag{29}$$

Since this simplex is compact, there exists a subsequence along which $q_{n,i} \to q_i$ for every i; we relabel it by n. Passing to the limit gives

$$\sum_{i=1}^{m} q_i v_i = 0, \quad q_i \geq 0, \sum_{i=1}^{m} q_i = 1 \tag{30}$$

Since $\sum_{i=1}^{m} q_i = 1$, there exists some j such that $q_j > 0$. Furthermore, for every i such that $q_i > 0$, we have $q_{n,i} \to q_i > 0$, and hence $a_{n,i} = A_n q_{n,i} > 0$ for sufficiently large n.
Now define

$$t_n = \min_{i \colon q_i > 0} \frac{a_{n,i}}{q_i} > 0. \tag{31}$$

Consider the new coefficients

$$a'_{n,i} = a_{n,i} - t_n q_i. \tag{32}$$

By the definition of t_n, we have $a'_{n,i} \geq 0$ for all i. Furthermore,

$$\sum_{i=1}^{m} a'_{n,i} v_i = \sum_{i=1}^{m} (a_{n,i} - t_n q_i) v_i \tag{33}$$

$$= u_n - t_n \sum_{i=1}^{m} q_i v_i \tag{34}$$

$$= u_n. \tag{35}$$

Therefore, $(a'_{n,i})$ is another nonnegative representation of u_n. However,

$$\sum_{i=1}^{m} a'_{n,i} = \sum_{i=1}^{m} a_{n,i} - t_n \sum_{i=1}^{m} q_i \tag{36}$$

$$= A_n - t_n \tag{37}$$

$$< A_n, \tag{38}$$

contradicting the minimality of A_n. Therefore, (A_n) must be bounded.
Since $0 \leq a_{n,i} \leq A_n$ for all i, the sequence of coefficient vectors $(a_{n,1}, \dots, a_{n,m})$ is bounded in $\mathbb{R}^m$. By Bolzano-Weierstrass, there exists a convergent subsequence such that

$$a_{n,i} \to a_i \geq 0 \qquad \forall i. \tag{39}$$

Taking the limit in $u_n = \sum_{i=1}^m a_{n,i} v_i$ gives

$$u = \sum_{i=1}^m a_i v_i \in C. \tag{40}$$

Therefore, C is sequentially closed, and hence closed in $\mathbb{R}^n$.
Since we have shown that C is a closed convex cone, by the separating hyperplane theorem, there exists a nonzero linear functional $\ell \in V^\dagger$ and $\alpha \in \mathbb{R}$ such that

$$\ell(v) < \alpha \leq \ell(c) \qquad \forall c \in C. \tag{41}$$

Since $0 \in C$, we have

$$\ell(v) < \alpha \leq \ell(0) = 0, \tag{42}$$

and hence $\ell(v) < 0$. We claim that $\ell(c) \geq 0$ for all $c \in C$. Suppose otherwise, so that there exists $c \in C$ with $\ell(c) < 0$. Since C is a cone, $tc \in C$ for every $t \geq 0$. Thus,

$$t\ell(c) = \ell(tc) \geq \alpha \qquad \forall t \geq 0. \tag{43}$$

Taking $t \to \infty$ gives a contradiction, since $t\ell(c) \to -\infty$. Therefore, $\ell(c) \geq 0$ for all $c \in C$. In particular, since $v_i \in C$,

$$\ell(v_i) \geq 0 \quad \forall i, \qquad \ell(v) < 0. \tag{44}$$

In the rest of this section, vector inequalities are entry-wise. Elements of $\mathbb{R}^n$ are column vectors, and elements of the dual space $(\mathbb{R}^n)^\dagger$ are row vectors, so that a functional λ acts on r by the matrix product λr.

Corollary 1.4 (Farkas' Lemma, Equality Form)

Let $W \in \mathbb{R}^{n \times m}$ be the matrix whose columns are the generators[a] of a cone. For any $r \in \mathbb{R}^n$, the following are equivalent.

1. *Primal.* There exists an $a \in \mathbb{R}^m$ such that

$$r = Wa, \quad a \geq 0 \tag{45}$$

   That is, r lies in the *cone* generated by W.
2. *Dual.* For every row vector $\lambda \in \mathbb{R}^{1 \times n}$ satisfying $\lambda W \geq 0$, it is the case that

$$\lambda r \geq 0 \tag{46}$$

[a] Not necessarily linearly independent.

Proof. Obvious from theorem above.

Now here is a variant that relaxes the primal condition (the equality $r = Wa$ becomes the inequality $r \geq Wa$). On the dual side, we must now check $\lambda r \geq 0$ only for the λ that also satisfy $\lambda \geq 0$. This is a smaller set of λ's to check, so the dual condition is weaker, which matches the weaker primal condition.

Theorem 1.5 (Farkas' Lemma, Inequality Form)

Let $W \in \mathbb{R}^{n\times m}$ be the matrix whose columns are the generators[a] of a cone. For any $r \in \mathbb{R}^n$, the following are equivalent.

1. *Primal.* There exists an $a \in \mathbb{R}^m$ such that

$$r \geq Wa, \quad a \geq 0 \tag{47}$$

2. *Dual.* For every row vector $\lambda \in \mathbb{R}^{1\times n}$ satisfying $\lambda W \geq 0$ and $\lambda \geq 0$, it is the case that $\lambda r \geq 0$.

[a] Not necessarily linearly independent.

Proof. We prove bidirectionally.

1. ($\rightarrow$). Suppose there exists $a \geq 0$ such that $r \geq Wa$. For every $\lambda \geq 0$ satisfying $\lambda W \geq 0$, we have

$$\lambda r \geq \lambda W a \geq 0. \tag{48}$$

2. ($\leftarrow$). Suppose that for every $\lambda \geq 0$ satisfying $\lambda W \geq 0$, we have $\lambda r \geq 0$. Observe that the primal condition can be rewritten by introducing a slack variable $s \in \mathbb{R}^n_{\geq 0}$:

$$r \geq Wa, \quad a \geq 0 \iff r = Wa + s, \quad a, s \geq 0 \tag{49}$$

$$\iff r = \begin{pmatrix} W & I_n \end{pmatrix} \begin{pmatrix} a \\ s \end{pmatrix}, \quad \begin{pmatrix} a \\ s \end{pmatrix} \geq 0. \tag{50}$$

Define the augmented matrix

$$\widetilde{W} = \begin{pmatrix} W & I_n \end{pmatrix} \in \mathbb{R}^{n\times(m+n)}. \tag{51}$$

By the dual formulation of positive span, there exists a nonnegative coefficient vector b satisfying $r = \widetilde{W}b$ if and only if

$$\lambda\widetilde{W} \geq 0 \implies \lambda r \geq 0 \tag{52}$$

for every $\lambda \in \mathbb{R}^{1\times n}$. But

$$\lambda\widetilde{W} \geq 0 \iff \begin{pmatrix} \lambda W & \lambda I_n \end{pmatrix} \geq 0 \tag{53}$$

$$\iff \lambda W \geq 0, \quad \lambda \geq 0. \tag{54}$$

By assumption, every such λ satisfies $\lambda r \geq 0$. Therefore, there exists $b \geq 0$ such that

$$r = \widetilde{W}b. \tag{55}$$

Writing $b = (a^\top, s^\top)^\top$, we obtain

$$r = Wa + s, \qquad a, s \geq 0. \tag{56}$$

Hence $r \geq Wa$ with $a \geq 0$.

Definition 1.1 (Primal and Dual Feasibility)

Let $W \in \mathbb{R}^{n\times m}$, $r \in \mathbb{R}^n$, and $p \in \mathbb{R}^{1\times m}$. Consider the following two optimization problems.

1. *Primal.* The **primal problem** is

$$S = \sup_{a\in\mathbb{R}^m} \{pa \colon Wa \leq r,\ a \geq 0\}. \tag{57}$$

A vector $a \in \mathbb{R}^m$ satisfying $Wa \leq r, \qquad a \geq 0$ is called **primal feasible**.

2. *Dual.* The **dual problem** is

$$s = \inf_{\lambda \in \mathbb{R}^{1\times n}} \{\lambda r \colon \lambda W \geq p,\ \lambda \geq 0\}. \tag{58}$$

A row vector $\lambda \in \mathbb{R}^{1\times n}$ satisfying $\lambda W \geq p, \qquad \lambda \geq 0$ is called **dual feasible**.
For each problem, if the feasible set is nonempty, then the problem is called **feasible**.

In case the motivation is still not clear, we can do a manual derivation of the dual from the primal.

Example 1.1 (Manual Derivation of Dual from Primal)

Given a primal problem such as

$$\max 2x + y \text{ subject to } \begin{cases} x + y \leq 3 \\ x - y \leq 1 \\ x, y \geq 0 \end{cases} \tag{59}$$

we can just find *one* solution satisfying the constraints and use it as a lower bound on the optimum. Say we find one, e.g. $x = y = 0$, which does satisfy all constraints. Now if we want to find an *upper bound* on the value of the solution, we can use duality. We create new variables $a, b \geq 0$, and observe that

$$x + y \leq 3 \implies a(x + y) \leq 3a \tag{60}$$
$$x - y \leq 1 \implies b(x - y) \leq b \tag{61}$$

Thus by adding the two we have $(a + b)x + (a - b)y \leq 3a + b$.
Now if we suppose $a + b \geq 2$ and $a - b \geq 1$ (which are the coeffients of $2x + y$), then since $x, y \geq 0$, we have

$$2x + y \leq (a + b)x + (a - b)y \leq 3a + b \tag{62}$$

so any a, b satisfying the above inequalities give an upper bound of $3a + b$ on the value of the optimal solution. Say we have $a = 2, b = 1$, which does satisfy. Then $3a + b = 7$ is an upper bound of $2x + y$. Therefore, to get the tightest upper bound, we can minimize the dual problem

$$\min 3a + b \text{ subject to } \begin{cases} a + b \geq 2 \\ a - b \geq 1 \\ a, b \geq 0 \end{cases} \tag{63}$$

It should be clear from our line of results that point that we wish to make is that the primal and dual are two reformulations of the same problem, and by solving one, we can solve the other.

Lemma 1.6 (Dual of Dual is Primal)

Given the primal and dual problems above,

$$S = \sup_{a \in \mathbb{R}^m} \{pa \colon Wa \leq r,\ a \geq 0\}, \quad s = \inf_{\lambda \in \mathbb{R}^{1\times n}} \{\lambda r \colon \lambda W \geq p,\ \lambda \geq 0\}. \tag{64}$$

we can reformulate the dual problem as a primal problem, and the dual of the dual is the primal.

Proof. Let $\nu = \lambda^T$. Then, we can reformulate the dual as a primal problem

$$s = \inf_{\nu \in \mathbb{R}^n} \{r^T \nu \colon W^T \nu \geq p^T, \nu \geq 0\} \tag{65}$$

$$= \sup_{\nu \in \mathbb{R}^n} \{(-r)^T \nu : (-W)^T \nu \leq (-p)^T, \nu \geq 0\} \tag{66}$$

The dual of this is

$$\inf_{\phi \in \mathbb{R}^{1 \times n}} \{\phi(-p)^T \colon \phi(-W)^T \geq (-r)^T, \phi \geq 0\} \tag{67}$$

We can reformulate it as a primal again, and substituting $\mu = \phi^T$ gives us

$$\sup_{\phi \in \mathbb{R}^{1 \times n}} \{\phi p^T \colon W\phi^T \leq r, \phi \geq 0\} = \sup_{\mu \in \mathbb{R}^n} \{p\mu \colon W\mu \leq r, \mu \geq 0\} \tag{68}$$

which is exactly the first primal problem.

Lemma 1.7 (Weak Duality)

Let $W \in \mathbb{R}^{n \times m}$, $r \in \mathbb{R}^n$, and $p \in \mathbb{R}^{1 \times m}$. For any primal feasible vector a and dual feasible vector λ, we have

$$pa \leq \lambda r. \tag{69}$$

Consequently, if both problems are feasible,

$$S \leq s. \tag{70}$$

Proof. Since $\lambda W \geq p$ and $a \geq 0$,

$$pa \leq \lambda W a. \tag{71}$$

Furthermore, since $Wa \leq r$ and $\lambda \geq 0$,

$$\lambda W a \leq \lambda r. \tag{72}$$

Combining these inequalities gives

$$pa \leq \lambda W a \leq \lambda r. \tag{73}$$

Taking the supremum over all primal feasible a and the infimum over all dual feasible λ gives $S \leq s$.

Now, we can establish the equivalence of the primal and dual problems.

Theorem 1.8 (Strong Duality)

Let $W \in \mathbb{R}^{n \times m}$, $r \in \mathbb{R}^n$, and $p \in \mathbb{R}^{1 \times m}$, with the conventions $\sup \emptyset = -\infty$ and $\inf \emptyset = +\infty$. Then exactly one of the following holds.

1. Both optimal values are finite and attained, i.e. there exist a primal feasible vector a^* and a dual feasible vector λ^* such that
$$pa^* = S = s = \lambda^* r. \tag{74}$$
2. The primal is feasible and unbounded ($S = +\infty$), and the dual is infeasible.
3. The dual is feasible and unbounded ($s = -\infty$), and the primal is infeasible.
4. Both problems are infeasible.

In particular, if either problem has a finite optimal value, so does the other.

Proof. The four cases correspond to the four combinations of feasibility, so we treat each.

1. *Both feasible.* Fix a primal feasible a_0 and a dual feasible λ_0. We claim that there exist $a \geq 0$ and $\nu \geq 0$ (a column vector, $\nu = \lambda^\top$) such that

$$Wa \leq r, \qquad -W^\top \nu \leq -p^\top, \qquad -pa + r^\top \nu \leq 0. \tag{75}$$

That is, a is primal feasible, $\lambda = \nu^\top$ is dual feasible, and $pa \geq \lambda r$. Given this claim, weak duality gives $pa \leq S \leq s \leq \lambda r$, and combined with $pa \geq \lambda r$ all of these are equalities. Hence $S = s$ is finite and attained at $a^* = a$ and $\lambda^* = \lambda$.
To prove the claim, write the system as $Mz \leq b$ with $z = (a, \nu) \geq 0$, where

$$M = \begin{pmatrix} W & 0 \\ 0 & -W^\top \\ -p & r^\top \end{pmatrix}, \qquad b = \begin{pmatrix} r \\ -p^\top \\ 0 \end{pmatrix}. \tag{76}$$

By Farkas' lemma (inequality form), such a z exists if and only if every row vector $y = (y_1, y_2, t) \geq 0$, with $y_1 \in \mathbb{R}^{1\times n}$, $y_2 \in \mathbb{R}^{1\times m}$, $t \in \mathbb{R}$, satisfying $yM \geq 0$ also satisfies $yb \geq 0$. Computing,

$$yM \geq 0 \iff y_1 W \geq tp \text{ and } Wy_2^\top \leq tr, \qquad yb = y_1 r - py_2^\top. \tag{77}$$

Let $y \geq 0$ satisfy the left-hand conditions. There are two cases.

- $t > 0$. Dividing by t, the vector $\lambda = y_1/t$ is dual feasible and $a = y_2^\top/t$ is primal feasible. By weak duality, $pa \leq \lambda r$, and multiplying by t gives $y_1 r - py_2^\top \geq 0$.
- $t = 0$. Then $y_1 W \geq 0$ and $Wy_2^\top \leq 0$. Using $y_1 \geq 0$, $Wa_0 \leq r$, $a_0 \geq 0$,

$$y_1 r \geq y_1 W a_0 \geq 0. \tag{78}$$

Using $y_2 \geq 0$, $\lambda_0 W \geq p$, $\lambda_0 \geq 0$,

$$py_2^\top \leq \lambda_0 W y_2^\top \leq 0. \tag{79}$$

Together, $y_1 r - py_2^\top \geq 0$.

This proves the claim.

2. *Primal feasible, dual infeasible.* That the dual is infeasible means there is no $\nu \geq 0$ with $-W^\top \nu \leq -p^\top$. By Farkas' lemma (inequality form) applied to the matrix $-W^\top$ and the vector $-p^\top$, there exists a row vector $y \geq 0$ with $-yW^\top \geq 0$ and $-yp^\top < 0$. Setting $d = y^\top \in \mathbb{R}^m$, this says $d \geq 0$, $Wd \leq 0$, and $pd > 0$. For a primal feasible a_0 and any $\tau \geq 0$, the vector $a_0 + \tau d$ is primal feasible, since $W(a_0 + \tau d) \leq Wa_0 \leq r$. Its objective value is $pa_0 + \tau pd \to \infty$, so $S = +\infty$.
3. *Dual feasible, primal infeasible.* That the primal is infeasible means there is no $a \geq 0$ with $Wa \leq r$. By Farkas' lemma (inequality form) applied directly, there exists $\lambda \geq 0$ with $\lambda W \geq 0$ and $\lambda r < 0$. For a dual feasible λ_0 and any $\tau \geq 0$, the vector $\lambda_0 + \tau\lambda$ is dual feasible, and its objective value is $\lambda_0 r + \tau \lambda r \to -\infty$, so $s = -\infty$.
4. *Both infeasible.* Nothing to prove.

The four cases are mutually exclusive since they have distinct feasibility patterns.

Corollary 1.9 (Primal-Dual Optimality Criterion)

A primal feasible vector a^* and a dual feasible vector λ^* are both optimal if and only if

$$pa^* = \lambda^* r. \tag{80}$$

Proof. We prove bidirectionally.

1. ($\rightarrow$). By strong duality,
$$pa^* = S = s = \lambda^* r. \tag{81}$$
2. ($\leftarrow$). By weak duality, for any primal feasible a and dual feasible λ,
$$pa \leq \lambda^* r = pa^* \leq \lambda r. \tag{82}$$
Therefore, a^* maximizes the primal objective and λ^* minimizes the dual objective.

Weak duality is elementary and follows directly from the inequalities. Strong duality is the substantial theorem, and it is exactly where Farkas' lemma is needed. Note that the optimality criterion above is a *certificate*: to prove that a feasible a^* is optimal, it suffices to exhibit a dual feasible λ^* with the same objective value.

Example 1.2 (Production Planning and Resource Pricing)

Suppose a furniture factory produces tables and chairs. Each table requires 2 units of wood and 1 hour of labor, generating a profit of \$50. Each chair requires 1 unit of wood and 2 hours of labor, generating a profit of \$40. The factory has 100 units of wood and 80 hours of labor available.
Let $a_1, a_2 \geq 0$ denote the quantities of tables and chairs produced, respectively. We define
$$W = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \qquad r = \begin{pmatrix} 100 \\ 80 \end{pmatrix}, \qquad p = \begin{pmatrix} 50 & 40 \end{pmatrix}. \tag{83}$$

1. *Primal.* The factory wishes to maximize its profit subject to its resource constraints:
$$S = \sup_{a \geq 0} \{pa \colon Wa \leq r\}. \tag{84}$$
Equivalently,
$$\begin{aligned} \text{maximize} \quad & 50a_1 + 40a_2 && (85)\\ \text{subject to} \quad & 2a_1 + a_2 \leq 100, && (86)\\ & a_1 + 2a_2 \leq 80, && (87)\\ & a_1, a_2 \geq 0. && (88) \end{aligned}$$
2. *Dual.* Let $\lambda_1, \lambda_2 \geq 0$ represent the implicit prices of wood and labor.[a] The total resource value required to produce each product must be at least its profit. Therefore,
$$s = \inf_{\lambda \geq 0} \{\lambda r \colon \lambda W \geq p\}. \tag{89}$$
Equivalently,
$$\begin{aligned} \text{minimize} \quad & 100\lambda_1 + 80\lambda_2 && (90)\\ \text{subject to} \quad & 2\lambda_1 + \lambda_2 \geq 50, && (91)\\ & \lambda_1 + 2\lambda_2 \geq 40, && (92)\\ & \lambda_1, \lambda_2 \geq 0. && (93) \end{aligned}$$

Solving the primal problem gives
$$a^* = \begin{pmatrix} 40 \\ 20 \end{pmatrix}, \qquad S = pa^* = 2800. \tag{94}$$

Solving the dual problem gives

$$\lambda^* = \begin{pmatrix} 20 & 10 \end{pmatrix}, \qquad s = \lambda^* r = 2800. \tag{95}$$

Therefore,

$$S = s = 2800, \tag{96}$$

verifying strong duality.

[a]Suppose a buyer wants to purchase all your wood and labor instead of letting you make furniture. What is the lowest price the buyer could offer that you would still accept? Let λ_1 be the price per unit of wood and λ_2 the price per hour of labor. The buyer's total cost is $100\lambda_1 + 80\lambda_2$ and the buyer wants to minimize it. This is the dual objective. You would refuse any offer that pays less than you could earn by making furniture. A table uses 2 wood and 1 labor hour and earns \$50, so you need $2\lambda_1 + \lambda_2 \geq 50$. A chair needs $\lambda_1 + 2\lambda_2 \geq 40$. These are the dual constraints $\lambda W \geq p$. Prices can't be negative, which gives $\lambda \geq 0$.

Example 1.3 (Primal Feasible, Dual Infeasible)

An arbitrage is a trading strategy that requires no net investment and cannot lose money, but has a positive profit in at least one outcome. For example, suppose a stock ABC trades at \$100, and a European call option on ABC with strike \$90 expiring in one year trades at \$8. Since the right to buy at \$90 a stock worth \$100 is worth at least \$10, the option is underpriced. Consider the following trade today: buy one call for \$8 and short sell one share of ABC for \$100, for a net cash inflow of \$92. At expiration, with stock price S_T,

- if $S_T \geq 90$, exercise the call, buy the share at \$90 and return it to close the short, leaving a profit of $92 - 90 = \$2$;
- if $S_T < 90$, let the call expire, buy the share on the market at S_T and return it, leaving a profit of $92 - S_T > \$2$.

In every outcome we keep at least \$2 without having invested anything (ignoring interest). Doing the same trade on t contracts multiplies the profit by t.
Let

$$W = \begin{pmatrix} 1 & -1 \end{pmatrix}, \qquad r = 1, \qquad p = \begin{pmatrix} 1 & 1 \end{pmatrix}, \tag{97}$$

so that the primal is

$$\begin{aligned} \text{maximize} \quad & a_1 + a_2 && (98)\\ \text{subject to} \quad & a_1 - a_2 \leq 1, && (99)\\ & a_1, a_2 \geq 0. && (100) \end{aligned}$$

The primal is feasible, since $a = (0, 0)$ satisfies the constraints. It is also unbounded: for any $t \geq 0$, the vector $a = (t, t)$ satisfies $Wa = 0 \leq 1$ and has objective value $2t \to \infty$.
The dual, with a single variable $\lambda \geq 0$, is

$$\begin{aligned} \text{minimize} \quad & \lambda && (101)\\ \text{subject to} \quad & \lambda \geq 1, && (102)\\ & -\lambda \geq 1, && (103)\\ & \lambda \geq 0. && (104) \end{aligned}$$

The first two constraints say $\lambda \geq 1$ and $\lambda \leq -1$, which cannot both hold, so the dual is infeasible. This is case 2 of the strong duality theorem. The vector $d = (1, 1)^\top$ is the direction from the proof: $d \geq 0$, $Wd = 0 \leq 0$, and $pd = 2 > 0$. Moving along d keeps the primal feasible while increasing the objective without bound. Weak duality is consistent with this: it says every dual feasible λ gives an upper bound on the primal, and here there is no such λ, so nothing stops the primal from growing.

Example 1.4 (Both Infeasible)

A problem is **infeasible** when its constraints contradict each other, so that no choice of variables satisfies all of them at once. This typically happens when constraints come from different sources that nobody checked against each other. For example, suppose a hospital staffs a cardiac ward and a general ward, and let a_1, a_2 be the numbers of nurses assigned to each, with the goal of staffing as many nurses as possible ($\max a_1 + a_2$). The nurses' union contract says that the cardiac ward may have at most one more nurse than the general ward,

$$a_1 - a_2 \leq 1, \tag{105}$$

while a new state safety regulation requires the cardiac ward to have at least two more nurses than the general ward,

$$a_1 - a_2 \geq 2. \tag{106}$$

No staffing plan satisfies both since adding the contract inequality $a_1 - a_2 \leq 1$ to the regulation written as $-a_1 + a_2 \leq -2$ gives $0 \leq -1$, which is absurd. Every staffing plan breaks one of the two rules, so the problem is infeasible. Let

$$W = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}, \qquad r = \begin{pmatrix} 1 \\ -2 \end{pmatrix}, \qquad p = \begin{pmatrix} 1 & 1 \end{pmatrix}. \tag{107}$$

1. *Primal.* The constraints $Wa \leq r$ read $a_1 - a_2 \leq 1$ and $-a_1 + a_2 \leq -2$, i.e. $a_1 - a_2 \leq 1$ and $a_1 - a_2 \geq 2$. These contradict each other, so the primal is infeasible. A certificate (from the inequality form of Farkas' lemma) is $\lambda = (1, 1)$, which satisfies $\lambda \geq 0$ and $\lambda W = (0, 0) \geq 0$, but $\lambda r = 1 - 2 = -1 < 0$.
2. *Dual.* The constraints $\lambda W \geq p$ with $\lambda \geq 0$ read

$$\lambda_1 - \lambda_2 \geq 1, \qquad -\lambda_1 + \lambda_2 \geq 1. \tag{108}$$

   Adding them gives $0 \geq 2$, so the dual is infeasible. A certificate is $d = (1, 1)^\top$, which satisfies $d \geq 0$, $Wd = (0, 0)^\top \leq 0$, and $pd = 2 > 0$.

This is case 4. With the conventions $\sup \emptyset = -\infty$ and $\inf \emptyset = +\infty$, we get $S = -\infty$ and $s = +\infty$. The two values differ, so strong duality gives no information in this case. Note that the vector d would make the primal unbounded *if* the primal had a feasible point, but it does not.

1.2 Complementary Slackness and Sensitivity

Strong duality has two useful consequences. The first tells us exactly how an optimal primal and dual pair must interact.

Theorem 1.10 (Complementary Slackness)

Let a be primal feasible and λ be dual feasible. Then a and λ are both optimal if and only if

$$\lambda(r - Wa) = 0 \qquad \text{and} \qquad (\lambda W - p)a = 0. \tag{109}$$

Proof. We have the identity

$$\lambda r - pa = \underbrace{\lambda(r - Wa)}_{\geq 0} + \underbrace{(\lambda W - p)a}_{\geq 0}. \tag{110}$$

Both terms are nonnegative by feasibility. By the optimality criterion, a and λ are both optimal if and only if $\lambda r - pa = 0$, which holds if and only if both nonnegative terms vanish.

Since every term in these two inner products is nonnegative, this is equivalent to the entrywise statements

$$\lambda_i > 0 \implies (Wa)_i = r_i, \qquad \text{(a priced resource is fully used)} \tag{111}$$
$$a_j > 0 \implies (\lambda W)_j = p_j. \qquad \text{(a produced good has zero profit after pricing)} \tag{112}$$

Example 1.5 (Complementary Slackness for Furniture, No Slack)

For the furniture example above, $a^* = (40, 20)^\top$ uses all of the wood and labor, $Wa^* = (100, 80)^\top = r$, and $\lambda^* W = (50, 40) = p$. So complementary slackness holds with $\lambda^* > 0$ and $a^* > 0$, with no slack in either place. We can also run this in reverse, to *compute* the dual solution from the primal one. Suppose we have only solved the primal and found $a^* = (40, 20)^\top$, and we do not yet know λ^*. Both a_1^* and a_2^* are positive, so complementary slackness forces both dual constraints to be tight, $(\lambda^* W)_j = p_j$ for $j = 1, 2$:

$$2\lambda_1 + \lambda_2 = 50, \tag{113}$$
$$\lambda_1 + 2\lambda_2 = 40. \tag{114}$$

This is a 2×2 linear system, and its solution is $\lambda^* = (20, 10)$. It remains to check that this λ^* is actually dual feasible and optimal: indeed $\lambda^* \geq 0$, $\lambda^* W = (50, 40) \geq p$, and $\lambda^* r = 100 \cdot 20 + 80 \cdot 10 = 2800 = pa^*$, so by the optimality criterion λ^* is optimal. Had a constraint been slack at a^* (say $(Wa^*)_i < r_i$), complementary slackness would instead give $\lambda_i = 0$, which removes an unknown.

In general, if we guess the set B of positive primal variables (the optimal basis in the language of the simplex method below), the equations $(\lambda W)_j = p_j$ for $j \in B$ and $\lambda_i = 0$ for the slack constraints form a linear system for λ^*, which is then verified for feasibility as above.

The second consequence is that the dual variables are marginal prices.

Theorem 1.11 (Dual Variables are Maringal/Shadow Prices)

Suppose the dual is feasible, and let $S(r) = \sup_{a \in \mathbb{R}^m} \{pa \colon Wa \leq r, a \geq 0\}$ denote the optimal value of the primal as a function of the resource vector r, with W and p fixed. Then,

1. On the set of r for which the primal is feasible, $S(r)$ is finite, concave, and piecewise linear in r.
2. If λ^* is an optimal dual solution for r, then

$$S(r') \leq S(r) + \lambda^*(r' - r) \qquad \forall r'. \tag{115}$$

That is, λ^* is a supergradient of S at r. In particular, wherever S is differentiable, $\partial S / \partial r_i = \lambda_i^*$.

Proof. The dual feasible set $D = \{\lambda \geq 0 \colon \lambda W \geq p\}$ does not depend on r. By strong duality, for every r with the primal feasible,

$$S(r) = \inf_{\lambda \in D} \lambda r, \tag{116}$$

which is a pointwise infimum of linear functions of r, and hence concave. It is finite since D and the primal are nonempty, and it is piecewise linear because D is a polyhedron whose infimum is attained at one of its finitely many vertices (it contains no lines since $\lambda \geq 0$). For the supergradient inequality, $\lambda^* \in D$ is dual feasible for every r', so $S(r') \leq \lambda^* r'$ by weak duality (or $S(r') = -\infty$), while $S(r) = \lambda^* r$. Subtracting gives the claim.

In the furniture example, $\lambda^* = (20, 10)$, so one extra unit of wood is worth \$20 and one extra hour of labor is worth \$10. Indeed with 101 units of wood, solving the constraints gives $a = (122/3, 59/3)$ and profit $2820 = 2800 + 20$.

1.3 Geometry of Linear Programs

We now turn to algorithms. For the geometry it is convenient to use the **standard form**, which is related to the canonical form by adding slack variables. In this subsection (and the next) the notation resets: $A \in \mathbb{R}^{m\times n}$ with $\operatorname{rank} A = m$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^{1\times n}$, and the LP is

$$\min\{cx \colon Ax = b,\ x \geq 0\}. \tag{117}$$

To convert the canonical primal $\max\{pa \colon Wa \leq r, a \geq 0\}$, we introduce slacks $s \geq 0$ with $Wa + s = r$, and set

$$x = \begin{pmatrix} a \\ s \end{pmatrix}, \quad A = \begin{pmatrix} W & I_n \end{pmatrix}, \quad b = r, \quad c = \begin{pmatrix} -p & 0 \end{pmatrix}, \tag{118}$$

so that maximizing pa becomes minimizing cx. Writing the dual of the standard form is easy. For any feasible x and any row vector λ (of either sign) with $\lambda A \leq c$, we have $\lambda b = \lambda A x \leq cx$ since $x \geq 0$. So the dual of the standard form is $\max\{\lambda b \colon \lambda A \leq c\}$, with λ free, matching the remark above that equalities give free dual variables.

The feasible set $P = \{x \colon Ax = b, x \geq 0\}$ is a *polyhedron*, an intersection of finitely many half-spaces.

Definition 1.2 (Extreme Point, Basic Feasible Solution)

A point $x \in P$ is an **extreme point** if it cannot be written as $x = \theta y + (1-\theta)z$ with $y \neq z$ in P and $\theta \in (0,1)$. For a set $B \subset \{1,\dots,n\}$ of m indices with the columns A_j, $j \in B$, linearly independent, let A_B be the invertible matrix of these columns. The point x with $x_B = A_B^{-1} b$ and $x_j = 0$ for $j \notin B$ is the **basic solution** for the basis B. If $x \geq 0$ it is a **basic feasible solution** (BFS). The indices in B are **basic** and the rest are **nonbasic**.

A BFS is determined by its basis, and there are at most $\binom{n}{m}$ bases. So there are only finitely many BFSs.

Lemma 1.12 (Extreme Points are Basic Feasible Solutions)

A point $x \in P$ is an extreme point if and only if the columns $\{A_j \colon x_j > 0\}$ are linearly independent. Equivalently (since $\operatorname{rank} A = m$, a linearly independent set of columns can be extended to a basis), extreme points of P are precisely the BFSs.

Proof. Let $\operatorname{supp}(x) = \{j \colon x_j > 0\}$.
($\leftarrow$) Suppose $x = \theta y + (1-\theta)z$ with $y, z \in P$, $y \neq z$, and $\theta \in (0,1)$. Since $y, z \geq 0$ and $x_j = 0$ forces $\theta y_j + (1-\theta) z_j = 0$, we get $y_j = z_j = 0$ for $j \notin \operatorname{supp}(x)$. So $d = y - z \neq 0$ is supported in $\operatorname{supp}(x)$ and $Ad = 0$, which is a nontrivial linear dependence among the columns A_j, $j \in \operatorname{supp}(x)$. So if those columns are independent, x is extreme.
($\rightarrow$) Suppose there is $d \neq 0$ supported in $\operatorname{supp}(x)$ with $Ad = 0$. Since $x_j > 0$ on $\operatorname{supp}(x)$, for small $\varepsilon > 0$ both $x \pm \varepsilon d \geq 0$, and $A(x \pm \varepsilon d) = b$. Then x is the midpoint of $x + \varepsilon d$ and $x - \varepsilon d$, so it is not extreme.

Theorem 1.13 (Fundamental Theorem of Linear Programming)

Suppose $P \neq \emptyset$ and cx is bounded below on P. Then the minimum of cx over P is attained at a basic feasible solution. In particular, P has a BFS.

Proof. Let $x_0 \in P$. Among all $x \in P$ with $cx \leq cx_0$, choose one with the fewest nonzero entries. We claim that the columns A_j, $j \in \operatorname{supp}(x)$, are linearly independent, so that x is a BFS by the lemma. Suppose not. Then there is $d \neq 0$ supported in $\operatorname{supp}(x)$ with $Ad = 0$. Replacing d by $-d$ if necessary, we may assume $cd \leq 0$. Note that $x + td$ satisfies $A(x + td) = b$ for all t.

- If $d \geq 0$ and $cd < 0$, then $x + td \in P$ for all $t \geq 0$ and $c(x+td) \to -\infty$, contradicting boundedness.
- If $d \geq 0$ and $cd = 0$, replace d by $-d$. Since $d \neq 0$ it now has a negative entry, and $c(-d) = 0$. So we are in the next case.
- If d has a negative entry, let $t^* = \min\{x_j/(-d_j) \colon d_j < 0\} > 0$. Then $x' = x + t^* d \in P$, $cx' = cx + t^* cd \leq cx \leq cx_0$, and x' has strictly fewer nonzero entries than x (its support is contained in $\operatorname{supp}(x)$ and one more entry vanishes), contradicting the choice of x.

So x is a BFS with $cx \leq cx_0$. Since x_0 was arbitrary, $\inf_P cx = \min_{\mathrm{BFS}} cx$, and this minimum exists since there are finitely many BFSs.

This reduces an optimization over an infinite set to a search over finitely many vertices, but the number of vertices can still be exponentially large. The simplex method moves from vertex to adjacent vertex, never increasing the cost.

1.4 The Simplex Method

Fix a basis B with BFS x ($x_B = A_B^{-1} b \geq 0$, $x_N = 0$), where N denotes the nonbasic indices. Every feasible y satisfies $y_B = A_B^{-1}(b - A_N y_N)$, so its cost is

$$cy = c_B y_B + c_N y_N = c_B A_B^{-1} b + \underbrace{(c_N - c_B A_B^{-1} A_N)}_{\bar{c}_N} y_N = cx + \bar{c}_N y_N. \tag{119}$$

Define the **reduced cost** of index j as $\bar{c}_j = c_j - c_B A_B^{-1} A_j$, which is 0 for $j \in B$. Intuitively, $\bar{c}_j$ is the rate at which the cost changes if we increase the nonbasic variable x_j from zero and let the basic variables adjust to maintain $Ax = b$.

Theorem 1.14 (Optimality Test)

If $\bar{c}_j \geq 0$ for all j, then the BFS x is optimal. Moreover, $\lambda = c_B A_B^{-1}$ is then an optimal solution of the dual $\max\{\lambda b \colon \lambda A \leq c\}$.

Proof. From the display above, $cy = cx + \bar{c}_N y_N \geq cx$ for every feasible y, since $\bar{c}_N \geq 0$ and $y_N \geq 0$. For the dual, $\bar{c}_j = c_j - \lambda A_j \geq 0$ says exactly that $\lambda A \leq c$, so λ is dual feasible, and $\lambda b = c_B A_B^{-1} b = cx$. By weak duality (the optimality criterion), both are optimal.

So the simplex method terminates with a primal solution together with its own dual certificate. If instead some $\bar{c}_j < 0$, increasing x_j decreases the cost. Let $d = A_B^{-1} A_j$. Setting $x_j = t$ and keeping the other nonbasic variables zero gives $x_B(t) = x_B - td$. Either $d \leq 0$, so $x(t)$ is feasible for all $t \geq 0$ with cost $cx + t\bar{c}_j \to -\infty$ (the LP is unbounded), or we can increase t until the first basic variable hits 0. This gives the pivot.

Algorithm 1.1 (Simplex Method)

Given a basic feasible solution x with basis B.

1. *Pricing.* Compute $\lambda = c_B A_B^{-1}$ and $\bar{c}_j = c_j - \lambda A_j$ for $j \in N$. If $\bar{c} \geq 0$, stop: x is optimal. Otherwise choose an **entering** index j with $\bar{c}_j < 0$.
2. *Direction.* Compute $d = A_B^{-1} A_j$. If $d \leq 0$, stop: the LP is unbounded.
3. *Ratio test.* Compute $t^* = \min\{x_{B_i}/d_i \colon d_i > 0\}$ and let B_i be an index attaining the minimum; it is the **leaving** index.
4. *Update.* Replace x_B by $x_B - t^* d$, set $x_j = t^*$, and replace B_i by j in the basis. The new basis is again a basis, and x is again a BFS, with cost changed by $t^* \bar{c}_j \leq 0$. Return to step 1.

We make four remarks on the method.

1. *Termination.* If $t^* > 0$ at every step, the cost strictly decreases, so no basis is repeated and since

there are finitely many bases the method terminates. When some basic variable is already zero (a **degenerate** BFS), we can have $t^* = 0$ and the method may *cycle* through the same bases. Bland's rule (among the eligible choices, always pick the entering and leaving indices of smallest index) provably prevents cycling.

2. *Initialization.* A starting BFS comes from **Phase I**. Multiply rows by -1 so that $b \geq 0$ and solve $\min\{\mathbf{1}^\top s \colon Ax + s = b, x, s \geq 0\}$ starting from the obvious BFS $(x, s) = (0, b)$. The original LP is feasible if and only if the optimum is 0, and an optimal BFS with $s = 0$ gives a BFS of P (after pivoting out any zero-valued artificial variables that remain basic).
3. *Complexity.* Each pivot costs a few linear solves, and in practice the method needs a number of pivots that is roughly linear in m. However, the worst case is exponential: for Dantzig's rule (choose the most negative $\bar{c}_j$), the Klee–Minty cube is a deformed n-cube on which the method visits all 2^n vertices. Smoothed analysis (Spielman–Teng) explains why this does not happen on perturbed instances.
4. *Polynomial-time methods.* Khachiyan's ellipsoid method (1979) was the first polynomial-time algorithm for LP, although it is slow in practice. Karmarkar's interior-point method (1984) is both polynomial and practical, and is discussed below.

Example 1.6 (Simplex on the Furniture Problem)

Return to the production problem. In standard form with slacks s_1, s_2 and $x = (a_1, a_2, s_1, s_2)$,

$$\min\ -50a_1 - 40a_2 \quad \text{s.t.} \quad \begin{pmatrix} 2 & 1 & 1 & 0 \\ 1 & 2 & 0 & 1 \end{pmatrix} x = \begin{pmatrix} 100 \\ 80 \end{pmatrix}, \quad x \geq 0. \tag{120}$$

Start from the basis $\{s_1, s_2\}$, i.e. $x = (0, 0, 100, 80)$ with cost 0.

1. Here $\lambda = 0$, so $\bar{c} = (-50, -40, 0, 0)$. We choose a_1 to enter. Then $d = (2, 1)$ and the ratios are $100/2 = 50$ and $80/1 = 80$, so s_1 leaves with $t^* = 50$. The new BFS is $x = (50, 0, 0, 30)$ with cost -2500.
2. With basis $\{a_1, s_2\}$, $A_B = \begin{pmatrix} 2 & 0 \\ 1 & 1 \end{pmatrix}$ and $\lambda = c_B A_B^{-1} = (-25, 0)$. The reduced costs are $\bar{c}_{a_2} = -40 - (-25)(1) = -15$ and $\bar{c}_{s_1} = 0 - (-25) = 25$. We choose a_2 to enter. Then $d = A_B^{-1}(1, 2)^\top = (1/2, 3/2)$ and the ratios are $50/(1/2) = 100$ and $30/(3/2) = 20$, so s_2 leaves with $t^* = 20$. The new BFS is $x = (40, 20, 0, 0)$ with cost -2800.
3. With basis $\{a_1, a_2\}$, $A_B = W$ and $\lambda = c_B W^{-1} = (-20, -10)$. The reduced costs are $\bar{c}_{s_1} = 20$ and $\bar{c}_{s_2} = 10$, both nonnegative, so $x = (40, 20, 0, 0)$ is optimal.

The dual certificate is $\lambda = (-20, -10)$. Since we minimized $-pa$ rather than maximized pa, the sign flips, recovering the shadow prices $(20, 10)$ from before.

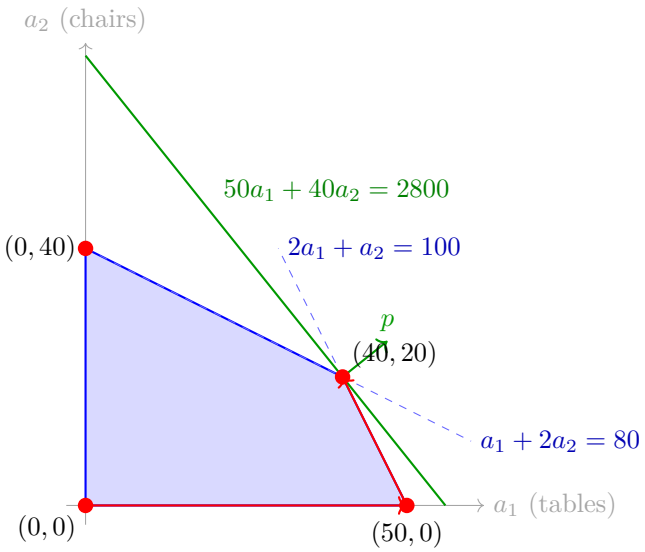


Figure 3: The feasible region of the furniture problem. Simplex starts at the origin and moves along edges (red) to the vertex $(40, 20)$, where the objective level line (green) touches the region last.

1.5 Interior-Point Methods and the Lagrangian View

Simplex walks along the boundary of P. Interior-point methods instead travel through its interior. Replace the standard-form LP by the barrier problem

$$\min_x \; cx - \mu \sum_{j=1}^{n} \log x_j \quad \text{s.t.} \quad Ax = b, \tag{121}$$

for $\mu > 0$. The logarithm forces $x > 0$, so the solution $x(\mu)$ is interior. The optimality conditions of this strictly convex problem are

$$Ax = b, \qquad A^\top \lambda^\top + s = c^\top, \qquad x_j s_j = \mu \;\; \forall j, \qquad x, s > 0, \tag{122}$$

with the sign convention that $s = c^\top - A^\top \lambda^\top \geq 0$ is the dual slack. The curve $\mu \mapsto (x(\mu), \lambda(\mu), s(\mu))$ is the **central path**. For $\mu = 0$ these conditions are exactly primal feasibility, dual feasibility and *complementary slackness* $x_j s_j = 0$, so as $\mu \to 0$ the central path converges to an optimal primal-dual pair. Interior-point methods apply Newton's method to this system while decreasing μ, and they take polynomially many Newton steps. The duality gap along the path is

$$cx - \lambda b = x^\top s = n\mu, \tag{123}$$

which gives a certificate of how far we are from optimal at every iteration.

Finally, LP duality is a special case of Lagrangian duality, which is developed for general constrained problems in later sections. For the canonical primal, define the Lagrangian $L(a, \lambda) = pa + \lambda(r - Wa)$ for $a \geq 0$ and $\lambda \geq 0$. For fixed $a \geq 0$,

$$\inf_{\lambda \geq 0} L(a, \lambda) = \begin{cases} pa & Wa \leq r \\ -\infty & \text{otherwise,} \end{cases} \tag{124}$$

so $S = \sup_{a \geq 0} \inf_{\lambda \geq 0} L(a, \lambda)$. On the other hand, rearranging $L(a, \lambda) = \lambda r + (p - \lambda W)a$, for fixed $\lambda \geq 0$

$$\sup_{a \geq 0} L(a, \lambda) = \begin{cases} \lambda r & \lambda W \geq p \\ +\infty & \text{otherwise,} \end{cases} \tag{125}$$

so $s = \inf_{\lambda \geq 0} \sup_{a \geq 0} L(a, \lambda)$. Weak duality is the general inequality $\sup \inf \leq \inf \sup$, and strong duality says that this minimax inequality is an equality for LPs, i.e. that (a^*, λ^*) is a saddle point of L. Complementary slackness is then the statement that $L(a^*, \lambda^*) = pa^* = \lambda^* r$.

2 Smooth Optimization

Definition 2.1 (ϵ-Critical Point)

For differentiable function $f : \mathbb{R}^d \to \mathbb{R}$, $x \in \mathbb{R}^d$ is an **ϵ-critical point** if

$$\|\nabla f(x)\| \leq \epsilon \tag{126}$$

Definition 2.2 (μ-Strongly Convex)

A function $f : \mathbb{R}^d \to \mathbb{R}$ is **μ-strongly convex** if for all $w, y \in \mathbb{R}^d$,

$$f(y) \geq f(w) + \langle \nabla f(w), y - w \rangle + \frac{\mu}{2}\|y - w\|^2 \tag{127}$$

Note that f is convex when $\mu = 0$.

Theorem 2.1 (Taylor)

Let $f : \mathbb{R}^d \to \mathbb{R}$ be a smooth function with L-lipschitz gradients

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\| \tag{128}$$

Then,

$$|f(y) - f(x) - \langle \nabla f(x), y - x \rangle| \leq \frac{L}{2}\|y - x\|^2 \tag{129}$$

Proof.

With this result, we can see that

$$f(x) \leq f(w) + \langle \nabla f(w), x - w \rangle + \frac{L}{2}\|x - w\|^2 \tag{130}$$

This is useful since it upper-bounds a potentially complicated nonlinear function by a simple quadratic function.

An algorithmic design principle that we will follow is that instead of minimizing f directly, we iteratively minimize an approximation of f centered at the previous iterate. We will use this principle over and over again. Therefore, let's start with this iterative procedure.

$$w_{k+1} \leftarrow \operatorname*{argmin}_w \Phi_k(w) = \operatorname*{argmin}_w \left\{ \underbrace{\langle \nabla f(w_k), w - w_k \rangle}_{\text{linear approximation}} + \underbrace{\frac{1}{\eta_k}\|w - w_k\|^2}_{\text{anchor}} \right\} \tag{131}$$

Note that the quadratic term is a regularizer that prevents us from moving too far from the current point w_k. This actually turns out to be gradient descent when we look at the first order optimality conditions. Recall that at a local minimum of a differentiable function, the gradient must vanish. So,

$$0 = \nabla \Phi_k(w_{k+1}) = \nabla f(w_k) + \frac{1}{\eta_k}(w_{k+1} - w_k) \implies w_{k+1} = w_k - \eta_k \nabla f(w_k) \tag{132}$$

So η is sort of like the step size. Let's try to show that this is a good idea.

Lemma 2.2 (Descent Lemma)

Let f be L-smooth and $\eta \in (0, \frac{2}{L})$. Then,

$$f(w - \eta \nabla f(w)) \leq f(w) - \eta\big(1 - \frac{L\eta}{2}\big)\|\nabla f(w)\|^2 \tag{133}$$

Proof. This follows from Taylor's with $y = w - \eta f(w)$, since

$$\begin{aligned} f(y) - f(w) + \eta\|\nabla f(w)\|^2 &= f(y) - f(w) + \langle \nabla f(w), y - w\rangle && (134)\\ &\leq \frac{L}{2}\|y - w\|^2 && (135)\\ &= \frac{L\eta^2}{2}\|\nabla f(w)\|^2 && (136) \end{aligned}$$

Theorem 2.3 (Spiral)

Suppose that f is L-smooth and $\inf f > -\infty$. Consider the iterates

$$w_{k+1} \leftarrow w_k - \frac{1}{L}\nabla f(w_k) \tag{137}$$

Then,

$$\min_{k \in [T-1]} \|\nabla f(w_k)\| \leq \sqrt{\frac{2L\big(f(w_0) - \inf f\big)}{T}} \tag{138}$$

Proof.

Theorem 2.4 (Quadratic Growth)

Suppose that f is μ-strongly convex. Then, f has a unique minimizer w^*, and for all $w \in \mathbb{R}^d$,

$$\frac{\mu}{2}\|w - w^*\|^2 \leq f(w) - f(w^*) \tag{139}$$

Theorem 2.5 (Lightning)

Suppose that f is L-smooth and μ-strongly convex. Then, the iterates of GD satisfy

$$f(w_k) - f(w^*) \leq \left(1 - \frac{\mu}{L}\right)^k \big(f(w_0) - f(w^*)\big) \tag{140}$$

A simply computation yields that if we want to reach

$$f(w_T) - f(w^*) \leq \epsilon \tag{141}$$

then it suffices to take

$$T \geq \frac{L}{\mu} \ln\left(\frac{f(w_0) - f(w^*)}{\epsilon}\right) \tag{142}$$

Further quadratic growth ensures

$$\|w_T - w^*\| \leq \frac{2\epsilon}{\mu} \tag{143}$$

Note that for theorem spiral, we needed $T \geq \frac{2L}{\epsilon^2}\big(f(w_0) - f(w^*)\big)$ for $f(w_T) - f(w^*) \leq \epsilon$. If $\epsilon = 10^{-6}$, then this means that $e^{-2} = 10^1 2$ while $\ln(\frac{1}{\epsilon}) \approx 14$.

Thus, how fast GD converges is dictated by the ratio

$$\kappa = \frac{L}{\mu} \in [1, \infty] \tag{144}$$

known as the **condition number**. Therefore, for general smooth functions, we can merely find critical points at a rather slow rate $O(\epsilon^{-2})$. If we want to guarantee fast convergence towards a minimizer, we need additional geometric structure. For a long time, most of the "additional geometric structure" optimizers used to prove guarantees amounted to strong convexity. In the last couple of decades, that has changed.

Definition 2.3 (Polyak-Lojasiewicz Inequality)

A differentiable function $f : \mathbb{R}^d \to \mathbb{R}$ satisfies the **PL-inequality**[a] if there exists a $\mu > 0$ s.t.

$$\mu\big(f(x) - \inf f\big) \leq \frac{1}{2}\|\nabla f(x)\|^2 \text{ for all } x \in \mathbb{R}^d \tag{145}$$

[a] This was published in 1963 and was ignored until 2016.

This is a relatively strong aassumption, and in particular it yields that

$$\nabla f(x) = 0 \implies f(x) = \inf f \tag{146}$$

Nonetheless, it generalizes the notion of strong convexity.

Lemma 2.6 (Strong Convexity Implies PL-Inequality)

Suppose $f : \mathbb{R}^d \to \mathbb{R}$ is differentiable and μ-strongly convex. Then, it satisfies μ-PL. Further, there exists a nonconvex smooth function that satisfies μ-PL.

Proof.

Theorem 2.7 (Double Lightning)

Suppose that f is L-smooth and satisfies μ-PL. Then, the iterates of GD with $\eta_k = \frac{1}{L}$ satisfy

$$f(w_k) - \inf(f) \leq \left(1 - \frac{\mu}{L}\right)^k \big(f(w_0) - f(w^*)\big) \tag{147}$$

Theorem 2.8

Suppsoe that f is differentiable, $A \in \mathbb{R}^{m \times d}$ and $b \in \mathbb{R}^m$. Define

$$\varphi(x) = f(Ax + b) \tag{148}$$

Then, the following hold.

1. If f is L-smooth, then φ is $(L\|A\|_{\text{op}}^2)$-smooth.

2. If f satisfies μ-PL and A is onto, φ satisfies $(\mu\lambda_{\min}(AA^T))$-PL, where $\lambda_{\min} = \inf_{\|v\|=1} v^T AA^T v$.

Proof.

3 Gradient Methods

The first thing you learn about gradients in multivariate calculus is that they point in the step of steepest ascent. Generally, you can think of the function you're trying to minimize as a "landscape." This inspires a greedy approach to simply walk in this direction $\nabla f(x)$ to maximize a function (or walk in the opposite direction $-\nabla f(x)$). This gives the following.

Algorithm 3.1 (Gradient Descent)

Generally speaking, at every point you should point in the direction of steepest descent, and move in that direction. The only question that remains is: how far? This is manually adjusted by the *learning rate*.

Require: Function $f(x)$, initialization x_0, learning rate

procedure GradientDescent(f, x_0)
    $x \leftarrow x_0$
    while not converged **do**
        $x \leftarrow x - \eta \nabla f(x)$
    end while
    return x
end procedure

3.1 Newton-Raphson Method for Root Finding

Before we talk more about gradient descent methods, let's take a look at one of the simplest numerical root-finders.[2]

Theorem 3.1 (Convergence)

Given a differentiable function $f : \mathbb{R} \to \mathbb{R}$, the sequence (x_n) defined

$$x_n = x_{n-1} - \frac{f(x_{n-1})}{f'(x_{n-1})} \tag{149}$$

for any $x_0 \in \mathbb{R}$, converges to a root of f.

Proof. Need to verify this.

[2] Optimization and root finding are very similar, since to optimize f you should first find a root of f'.

Algorithm 3.2 (Newton-Raphson Method)

Require: Function $f(x)$, initial guess x_0
procedure NEWTONRAPHSON($f, f', x_0, \epsilon, N_{max}$)
  $x \leftarrow x_0$
  while not converged **do**
    if $f'(x) = 0$ **then**
      Retry with another x_0.
    end if
    $x \leftarrow \frac{f(x)}{f'(x)}$
  end while
  return x
end procedure

3.2 Stochastic Gradient Descent

Now let's go back to gradient descent. Usually, in machine learning, we are trying to optimize a parameteric model $\mathcal{P} = \{\mathbb{P}_\theta \mid \theta \in \Theta\}$ over a dataset $\mathcal{D} = \{x_i\}_{i=1}^n$., For example, our estimator in the maximum-likelihood approach will be

$$\delta(\mathcal{D}) = \underset{\theta}{\operatorname{argmax}}\, L(\theta \mid \mathcal{D}) = \log p(\mathcal{D} \mid \theta) \tag{150}$$

for some loss function L.

If we assume that the samples are iid, then we can decompose the gradient as

$$\nabla_\theta \log p(\mathcal{D} \mid \theta) = \sum_i \nabla_\theta \log p(d_i \mid \theta) \tag{151}$$

which scales linearly with the size of a dataset. This is not scalable with extremely large datasets, and so we must remove this $O(n)$ term. Intuitively, we can think of approximating this gradient by taking a minibatch b of $\mathcal{D}$ and computing the gradient only across that minibatch.

Theorem 3.2 (Minibatch Gradient is an Unbiased Estimator)

Let us take a minibatch of b samples $\mathcal{B} \subset \mathcal{D}$ without replacement, where $b << D$. Then, our approximation of the gradient of the log likelihood

$$\nabla_\theta \log p(\mathcal{B} \mid \theta) \coloneqq \frac{1}{b} \sum_{x \in \mathcal{B}} \nabla_\theta \log p(x \mid \theta) \tag{152}$$

is an unbiased estimator of the true gradient $\nabla_\theta \log p(\mathcal{D} \mid \theta)$. That is, setting $\mathcal{M}$ as a random variable of samples over $\mathcal{D}$, we have

$$\mathbb{E}_{\mathcal{M}}[\nabla \mathcal{L}_{\mathcal{M}}(\mathbf{w})] = \nabla \mathcal{L}(\mathbf{w}) \tag{153}$$

Proof. We use linearity of expectation for all $\mathcal{M} \subset \mathcal{D}$ of size M.

This also has the additional advantage of saving memory. You don't have to load in the gradients for the whole dataset (which may be a few TB), and can allocate just enough memory for each batch (perhaps a few GB).

Algorithm 3.3 (Stochastic Gradient Descent)

Require: Function $L(\theta)$, initialization θ_0, learning rate, batch size b, dataset $\mathcal{D}$
procedure StochasticGradientDescent(f, θ_0, b)
  $\theta \leftarrow \theta_0$
  while not converged **do**
    Sample minibatch $\mathcal{B} \subset \mathcal{D}$.
    $\theta \leftarrow \theta - \eta \nabla_\theta \log p(\mathcal{B} \mid \theta)$
  end while
  return θ
end procedure

Example 3.1 (Linear Regression)

We have assumed knowledge of gradient descent in the back propagation step in the previous section, but let's revisit this by looking at linear regression. Given our dataset $\mathcal{D} = \{\mathbf{x}^(n), y^{(n)}\}$, we are fitting a linear model of the form

$$f(\mathbf{x}; \mathbf{w}, b) = \mathbf{w}^T \mathbf{x} + b \tag{154}$$

The squared loss function is

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{2} \sum_{n=1}^{N} \big(y - f(\mathbf{x}; \mathbf{w}, b)\big)^2 = \frac{1}{2} \sum_{n=1}^{N} \big(y - (\mathbf{w}^T \mathbf{x} + b)\big)^2 \tag{155}$$

If we want to minimize this function, we can visualize it as a d-dimensional surface that we have to traverse. Recall from multivariate calculus that the gradient of an arbitrary function $\mathcal{L}$ points in the steepest direction in which $\mathcal{L}$ increases. Therefore, if we can compute the gradient of $\mathcal{L}$ and step in the *opposite direction*, then we would make the more efficient progress towards minimizing this function (at least locally). The gradient can be solved using chain rule. Let us solve it with respect to $\mathbf{w}$ and b separately first. Beginners might find it simpler to compute the gradient element-wise.

$$\frac{\partial}{\partial w_j} \mathcal{L}(\mathbf{w}, b) = \frac{\partial}{\partial w_j} \left(\frac{1}{2} \sum_{n=1}^{N} \Big(f(\mathbf{x}^{(n)}; \mathbf{w}, b) - y^{(n)} \Big)^2 \right) \tag{156}$$

$$= \frac{1}{2} \sum_{n=1}^{N} \frac{\partial}{\partial w_j} \Big(f(\mathbf{x}^{(n)}; \mathbf{w}, b) - y^{(n)} \Big)^2 \tag{157}$$

$$= \frac{1}{2} \sum_{n=1}^{N} 2 \Big(f(\mathbf{x}^{(n)}) - y^{(n)} \Big) \cdot \frac{\partial}{\partial w_j} \big(f(\mathbf{x}^{(n)}; \mathbf{w}, b) - y^{(n)} \big) \tag{158}$$

$$= \frac{1}{2} \sum_{n=1}^{N} 2 \Big(f(\mathbf{x}^{(n)}) - y^{(n)} \Big) \cdot \frac{\partial}{\partial w_j} \big(\mathbf{w}^T \mathbf{x}^{(n)} + b - y^{(n)} \big) \tag{159}$$

$$= \sum_{n=1}^{N} \big(f(\mathbf{x}^{(n)}; \mathbf{w}, b) - y^{(n)} \big) \cdot x_j^{(n)} \qquad (\text{for } j = 0, 1, \ldots, d) \tag{160}$$

As for getting the derivative w.r.t. b, we can redo the computation and get

$$\frac{\partial}{\partial w_j} \mathcal{L}(\mathbf{w}, b) = \sum_{n=1}^{N} \big(f(\mathbf{x}^{(n)}; \mathbf{w}, b) - y^{(n)} \big) \tag{161}$$

and in the vector form, setting $\boldsymbol{\theta} = (\mathbf{w}, b)$, we can set

$$\nabla \mathcal{L}(\mathbf{w}) = \mathbf{X}^T(\hat{\mathbf{y}} - \mathbf{y}) \tag{162}$$
$$\nabla \mathcal{L}(b) = (\hat{\mathbf{y}} - \mathbf{y}) \cdot \mathbf{1} \tag{163}$$

where $\hat{\mathbf{y}}_n = f(\mathbf{x}^{(n)}; \mathbf{w}, b)$ are the predictions under our current linear model and $\mathbf{X} \in \mathbb{R}^{n \times d}$ is our design matrix. This can easily be done on a computer using a package like `numpy`.
Rather than updating the vector $\boldsymbol{\theta}$ in batches, we can apply **stochastic gradient descent** that works incrementally by updating $\boldsymbol{\theta}$ with each term in the summation. That is, rather than updating as a batch by performing the entire matrix computation by multiplying over N dimensions,

$$\nabla \mathcal{L}(\mathbf{w}) = \underbrace{\mathbf{X}^T}_{D \times N} \underbrace{(\hat{\mathbf{y}} - \mathbf{y})}_{N \times 1} \tag{164}$$

we can reduce this load by choosing a smaller subset $\mathcal{M} \subset \mathcal{D}$ of $M < N$ elements, which gives

$$\nabla \mathcal{L}_{\mathcal{M}}(\mathbf{w}) = \underbrace{\mathbf{X}_{\mathcal{M}}^T}_{D \times M} \underbrace{(\mathbf{y}\hat{\mathcal{M}} - \mathbf{y})}_{\mathcal{M}} {}_{M \times 1} \tag{165}$$

Even though these estimators are noisy, we get to do much more iterations and therefore have a faster net rate of convergence. But now we have an additional choice to make. What should our minibatch size be?

Heuristic 3.1 (Choosing Batch Size)

It really depends on your hardware, but generally,

1. A smaller batch size might mean more noisy estimates, and therefore may not converge. However, it tends to escape local minima better.
2. A high batch size means more exact estimates, but it tends to get stuck in local minima.

You want to make sure that the batches fit into memory.

3.3 Learning Rates and Schedulers

The algorithm may not converge if α (the step size) is too high, since it may overshoot. This can be solved by reducing the α with each step, using *schedulers*.

Ideally, we would want to have a variable step size $h(t)$ so that $h \to 0$ as $t \to +\infty$.

Algorithm 3.4 (Decay on Plateau Learning Rate)

Basically, this says that if the loss doesn't decrease for the past p epochs, then decrease the learning rate $\eta \leftarrow \gamma \cdot \eta$.

Require: Patience p, decay rate $0 < \gamma < 1$, initial θ_0, loss L
procedure LRDecayOnPlateau(p, γ, θ, η)
  best_loss $\leftarrow L(\theta)$
  bad_epochs $\leftarrow 0$
  while not converged **do**
    $\theta \leftarrow \theta - \eta \nabla L(\theta)$
    if $L(\theta) <$ best_loss **then**
      best_loss $\leftarrow L(\theta)$
      bad_epochs $\leftarrow 0$
    else
      bad_epochs $\leftarrow$ bad_epochs $+ 1$
    end if
    if bad_epochs $\geq p$ **then**
      $\eta \leftarrow \eta \cdot \gamma$
      bad_epochs $\leftarrow 0$
    end if
  end while
  return θ, η
end procedure

3.4 Momentum and Nesterov

Algorithm 3.5 (Stochastic Gradient Descent with Momentum)

This modifies vanilla SGD by keeping a running velocity term that accumulates past gradients, which smooths updates and helps escape sharp local minima.

Require: Learning rate η, momentum $0 \leq \mu < 1$, initial parameters θ_0, loss L
procedure SGDMomentum(η, μ, θ)
  $v \leftarrow 0$ ▷ Initialize velocity
  while not converged **do**
    $v \leftarrow \mu v + \nabla L(\theta)$ ▷ Decay prev. velocity and add in gradient (acceleration)
    $\theta \leftarrow \theta - \eta v$
  end while
  return θ
end procedure

Algorithm 3.6 (Stochastic Gradient Descent with Nesterov Momentum)

Nesterov momentum modifies standard momentum by computing the gradient at the *lookahead* position $\theta - \eta\mu v$, leading to faster convergence in practice.

Require: Learning rate η, momentum $0 \leq \mu < 1$, initial parameters θ_0, loss L
procedure SGDNesterov(η, μ, θ)
  $v \leftarrow 0$ ▷ Initialize velocity
  while not converged **do**
    $g \leftarrow \nabla L(\theta - \eta\mu v)$ ▷ Lookahead gradient
    $v \leftarrow \mu v + g$
    $\theta \leftarrow \theta - \eta v$
  end while
  return θ
end procedure

3.5 Block Coordinate Descent

Algorithm 3.7 (Block Gradient Descent)

Block Gradient Descent partitions the parameter vector θ into m disjoint blocks. At each iteration, it selects one block (cyclically or randomly) and updates only that block's parameters using the gradient, while keeping the other blocks fixed.

Require: Learning rate η, number of blocks m, partition $\theta = (\theta^{(1)}, \dots, \theta^{(m)})$, loss L
procedure BlockGradientDescent($\eta, \{\theta^{(j)}\}_{j=1}^{m}$)
  $t \leftarrow 0$
  while not converged **do**
    $j \leftarrow \text{SelectBlock}(t, m)$ ▷ e.g., cyclic: $j = (t \bmod m) + 1$
    $\theta^{(j)} \leftarrow \theta^{(j)} - \eta \nabla_{\theta^{(j)}} L(\theta^{(1)}, \dots, \theta^{(m)})$
    $t \leftarrow t + 1$
  end while
  return θ
end procedure

4 Subgradient Methods

Definition 4.1 (Convex Function)

A function $f : U \subset \mathbb{R}^n \to \mathbb{R}$ defined on a convex set U is convex if and only if for any $\mathbf{x}, \mathbf{y} \in U$

$$f\big(\lambda \mathbf{x} + (1-\lambda)\mathbf{y}\big) \leq \lambda f(\mathbf{x}) + (1-\lambda) f(\mathbf{y}) \tag{166}$$

Now if f is differentiable, then convexity is equivalent to

$$f(x) \geq f(y) + \nabla f(y)^T \cdot (x - y) \tag{167}$$

for all $x, y \in U$. That is, its local linear approximation always underestimates f.

It is well known that the mean square error of a linear map is convex. However, when we impose the L1 penalty, the loss function is now not differentiable at $\mathbf{0}$. Therefore, we must introduce the notion of a subgradient.

Definition 4.2 (Subgradient)

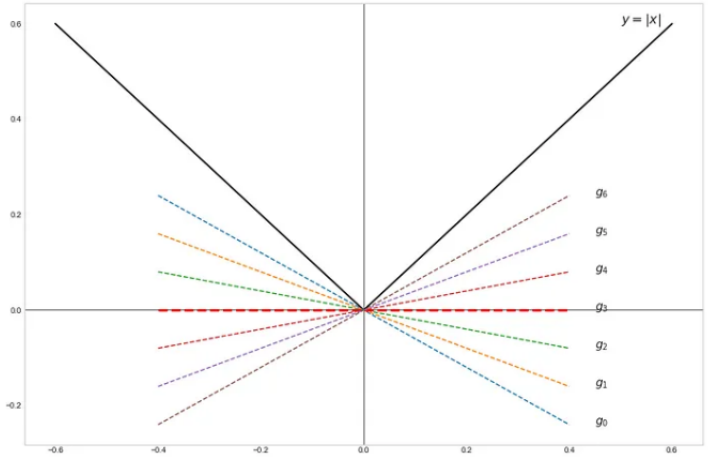
The subgradient of a convex function $f : U \subset \mathbb{R}^n \to \mathbb{R}$ is any linear map $\mathbf{A}(x) : \mathbb{R}^n \to \mathbb{R}$ such that

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \mathbf{A}(\mathbf{x})(\mathbf{y} - \mathbf{x}) \tag{168}$$

for any $\mathbf{y} \in U$. The set of all subgradients at $\mathbf{x}$ is called the **subdifferential** defined

$$\partial f(\mathbf{x}) = \{\mathbf{A} \in \mathbb{R}^n \mid \mathbf{A} \text{ is a subgradient of } f \text{ at } \mathbf{x}\} \tag{169}$$

The subgradient also acts as a linear approximation of f, but now at nondifferentiable points of convex functions, we have a set of linear approximations. It is clear that the subgradient at a differentiable point is uniquely the gradient ($\partial f(\mathbf{x}) = \{\nabla f(\mathbf{x})\}$), but for places like the absolute value, we can have infinite linear approximations.



Given the subdifferential, thus the optimality condition for any convex $\mathbf{f}$ (differentiable or not) is

$$f(\mathbf{x}^*) = \min_{\mathbf{x}} f(\mathbf{x}) \iff \mathbf{0} \in \partial f(\mathbf{x}^*) \tag{170}$$

known as the subgradient optimality condition, which clearly implies

$$f(\mathbf{y}) \geq f(\mathbf{x}^*) + \mathbf{0}^T(\mathbf{y} - \mathbf{x}^*) = f(\mathbf{x}^*) \tag{171}$$

Example 4.1

The subdifferential of the absolute value function $f(x) = |x|$ at any given x is

$$\partial f(x) = \begin{cases} 1 & \text{if } x > 0 \\ [-1, 1] & \text{if } x = 0 \\ -1 & \text{if } x < 0 \end{cases} \tag{172}$$

4.1 Proximal Gradient Descent

Definition 4.3 (Proximal Operator)

Given a lower semicontinuous convex function f mapping from Hilbert space X to $[-\infty, +\infty]$, its **proximal operator** associated with a point u is defined

$$\text{prox}_{f,\tau}(u) = \underset{x}{\text{argmin}} \left(f(x) + \frac{1}{2\tau} ||x - u||^2 \right) \tag{173}$$

where $\tau > 0$ is a parameter that scales the quadratic term. This is basically the point that minimizes the sum of $f(x)$ and the square of the Euclidean distance between x and u, scaled by $1/2\tau$.

Now given the loss function $L(\boldsymbol{\theta}) = L_{\text{obj}}(\boldsymbol{\theta}) + L_{\text{reg}}(\boldsymbol{\theta})$, we want to compute the proximal operator on the regularization loss and update that with the gradient of the smooth objective loss.

$$\boldsymbol{\theta}^{(k+1)} = \text{prox}_{L_{\text{reg}},\tau} \left[\boldsymbol{\theta}^{(k)} - \tau \nabla L_{\text{obj}}(\boldsymbol{\theta}^{(k)}) \right] \tag{174}$$

Let's compute the proximal operator of the L1 loss $h(\boldsymbol{\theta}) = \lambda ||\boldsymbol{\theta}||_1$. We can parameterize this loss by the λ, so we will use the notation $\text{prox}_{\lambda,\tau}$ rather than $\text{prox}_{h,\tau}$.

$$\begin{aligned} \text{prox}_{\lambda,\tau}(\mathbf{u}) &= \underset{\boldsymbol{\theta}}{\text{argmin}} \left(\lambda ||\boldsymbol{\theta}||_1 + \frac{1}{2\tau} ||\boldsymbol{\theta} - \mathbf{u}||_2^2 \right) \\ &= \underset{\boldsymbol{\theta}}{\text{argmin}} \left(\sum_{i=1}^{n} \lambda |\theta_i| + \frac{1}{2\tau} (\theta_i - u_i)^2 \right) \end{aligned}$$

These are separable functions that can be decoupled and optimized component-wise. So, we really just want to find

$$\theta_i^* = \underset{\theta_i}{\text{argmin}} \left(\lambda |\theta_i| + \frac{1}{2\tau} (\theta_i - u_i)^2 \right) \tag{175}$$

The sum of convex functions is convex, and so we should differentiate it and find where the gradient is 0 to optimize it.

1. When $\theta_i > 0$, then we minimize $\lambda\theta_i + \frac{1}{2\tau}(\theta_i - u_i)^2$, so taking the gradient and setting to 0 gives
$$\theta_i = u_i - \lambda\tau \tag{176}$$
subject to the constraint that $\theta_i > 0$, or equivalently, that $u_i > \lambda\tau$.
2. When $\theta_i < 0$, then we minimize $-\lambda\theta_i + \frac{1}{2\tau}(\theta_i - u_i)^2$, so taking the gradient and setting to 0 gives
$$\theta_i = u_i + \lambda\tau \tag{177}$$
subject to the constraint that $\theta_i < 0$, or equivalently, that $u_i < -\lambda\tau$.
3. When $\theta_i = 0$, then we minimize $\lambda|\theta_i| + \frac{1}{2\tau}(\theta_i - u_i)^2$, which doesn't have derivative at $\theta_i = 0$. So, we can compute the subdifferential of it to get
$$0 \in \partial \left(\lambda |\theta_i| + \frac{1}{2\tau} (\theta_i - u_i)^2 \right) = \lambda \partial(|\theta_i|) + \frac{1}{\tau}(\theta_i - u_i)$$

Now at $\theta_i = 0$, the subdifferential can be any value in $[-1, 1]$, and the above reduces to

$$0 \in \lambda[-1, 1] - \frac{1}{\tau} u_i \tag{178}$$

this is equivalent to saying that u_i/τ is contained in the interval $[-\lambda, \lambda]$, meaning that $u_i \in [-\lambda\tau, \lambda\tau]$. Ultimately we get that

$$\text{prox}_{\lambda,\tau}(u) = \begin{cases} u - \lambda\tau & \text{if } u > \lambda\tau \\ 0 & \text{if } |u| \leq \lambda\tau \\ u + \lambda\tau & \text{if } u < -\lambda\tau \end{cases} \tag{179}$$

which can be simplified to

$$\text{prox}_{\lambda,\tau}(u) = \text{sign}(u) \max\{|u| - \lambda\tau, 0) \tag{180}$$

5 Adaptive Gradient Methods

5.1 Adagrad

5.2 RMSProp and Adadelta

5.3 Adam

Adam and AdamW

6 Second-Order Optimizers

6.1 Newton's Method

Newton's method is an iterative algorithm for finding the roots of a differentiable function F. An immediate consequence is that given a convex C^2 function f, we can apply Newton's method to its derivative f' to get the critical points of f (minima, maxima, or saddle points), which is relevant in optimizing f. Given a C^1 function $f : D \subset \mathbb{R}^n \longrightarrow \mathbb{R}$ and a point $\mathbf{x}_k \in D$, we can compute its linear approximation as

$$f(\mathbf{x}_k + \mathbf{h}) \approx f(\mathbf{x}_k) + Df_{\mathbf{x}_k}\,\mathbf{h} = f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k) \cdot \mathbf{h} \tag{181}$$

where $Df_{\mathbf{x}_k}$ is the total derivative of f at $\mathbf{x}_k$ and $\mathbf{h}$ is a small n-vector. Discretizing this gives us our gradient descent algorithm as

$$\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \alpha\, f'(\mathbf{x}_k) \tag{182}$$

This linear function is unbounded, so we must tune the step size α accordingly. If α is too small, then convergence is slow, and if α is too big, we may overshoot the minimum. Netwon's method automatically tunes this α using the curvature information, i.e. the second derivative. If we take a second degree Taylor approximation

$$f(\mathbf{x}_k + \mathbf{h}) \approx f(\mathbf{x}_k) + Df_{\mathbf{x}_k}\,\mathbf{h} + \mathbf{h}^T\, Hf_{\mathbf{x}_k}\,\mathbf{h} \tag{183}$$

then we are guaranteed that this quadratic approximation of f has a minimum (existence and uniqueness can be proved), and we can calculate it to find our "approximate" minimum of f. We simply take the total derivative of this polynomial w.r.t. $\mathbf{h}$ and set it equal to the n-dimensional covector $\mathbf{0}$. This is equivalent to setting the gradient as $\mathbf{0}$, so

$$\begin{aligned}
\mathbf{0} &= \nabla_{\mathbf{h}}\big[f(\mathbf{x}_k) + Df_{\mathbf{x}_k}\,\mathbf{h} + \mathbf{h}^T\, Hf_{\mathbf{x}_k}\,\mathbf{h}\big](\mathbf{h}) \\
&= \nabla_{\mathbf{h}}[Df_{x_k}\mathbf{h}](\mathbf{h}) + \nabla_{\mathbf{h}}[\mathbf{h}^T\, Hf_{\mathbf{x}_k}\,\mathbf{h}](\mathbf{h}) \\
&= \nabla_{\mathbf{x}} f(\mathbf{x}_k) + Hf_{\mathbf{x}_k}\,\mathbf{h} \\
&\implies \mathbf{h} = -[Hf_{\mathbf{x}_k}]^{-1}\nabla_{\mathbf{x}} f(\mathbf{x}_k)
\end{aligned}$$

which results in the iterative update

$$\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - [Hf_{\mathbf{x}_k}]^{-1}\nabla_{\mathbf{x}} f(\mathbf{x}_k) \tag{184}$$

Note that we require $\mathbf{f}$ to be convex, so that Hf is positive definite. Since f is C^2, this implies Hf is also symmetric, implying invertibility by the spectral theorem. Note that Newton's method is very expensive, since we require the computation of the gradient, the Hessian, *and* the inverse of the Hessian, making the computational complexity of this algorithm to be $O(n^3)$. We can also add a smaller stepsize h to control stability.

Algorithm 1 Newton's Method

Require: Initial $\mathbf{x}_0$, Stepsize h (optional)
  for $t = 0$ to T until convergence **do**
    $g(\mathbf{x}_t) \leftarrow \nabla f(\mathbf{x}_t)$
    $H(\mathbf{x}_t) \leftarrow Hf_{\mathbf{x}_t}$
    $H^{-1}(\mathbf{x}_t) \leftarrow [H(\mathbf{x}_t)]^{-1}$
    $\mathbf{x}_{t+1} \leftarrow \mathbf{x}_t - h\, H^{-1}(\mathbf{x}_t)\, g(\mathbf{x}_t)$
  end for

6.2 Gauss Newton Method

7 Quasi-Newton Methods

7.1 Secant Method

Algorithm 7.1 (Secant Method)

The Secant Method approximates Newton's method by estimating the derivative using two most recent iterates instead of requiring $f'(x)$.

Require: Function $f(x)$, initial guesses x_0, x_1, tolerance ϵ, maximum iterations N_{max}

procedure SECANT($f, x_0, x_1, \epsilon, N_{max}$)
  $k \leftarrow 0$
  while $k < N_{max}$ **do**
    if $|f(x_1)| < \epsilon$ **then**
      return x_1
    end if
    $x_{new} \leftarrow x_1 - f(x_1)\dfrac{x_1 - x_0}{f(x_1) - f(x_0)}$
    $x_0 \leftarrow x_1$
    $x_1 \leftarrow x_{new}$
    $k \leftarrow k + 1$
  end while
  return Failure
end procedure

7.2 Davidon-Fletcher-Powell (DFP)

Algorithm 7.2 (DFP Quasi-Newton Method)

The DFP method is a quasi-Newton optimization algorithm that maintains an approximation H_k of the inverse Hessian to update parameters efficiently.

Require: Objective $f(\theta)$, gradient ∇f, initial guess θ_0, tolerance ϵ

procedure DFP($f, \nabla f, \theta_0$)
  $H \leftarrow I$ ▷ Initialize inverse Hessian approximation
  while not converged **do**
    $g \leftarrow \nabla f(\theta)$
    $d \leftarrow -Hg$
    $\alpha \leftarrow \text{LineSearch}(f, \theta, d)$
    $\theta_{new} \leftarrow \theta + \alpha d$
    $s \leftarrow \theta_{new} - \theta$
    $y \leftarrow \nabla f(\theta_{new}) - g$
    $H \leftarrow H + \frac{ss^T}{s^T y} - \frac{Hyy^T H}{y^T H y}$
    $\theta \leftarrow \theta_{new}$
  end while
  return θ
end procedure

7.3 Broyden's Method

Algorithm 7.3 (Broyden's Method)

Broyden's method is a quasi-Newton method that updates an approximation B_k to the Jacobian matrix without computing derivatives directly.

Require: Function $F(\theta)$, initial guess θ_0, tolerance ϵ
procedure BROYDEN(F, θ_0)
  $B \leftarrow I$ ▷ Initial Jacobian approximation
  while not converged **do**
    $\Delta\theta \leftarrow -B^{-1}F(\theta)$
    $\theta_{new} \leftarrow \theta + \Delta\theta$
    $\Delta F \leftarrow F(\theta_{new}) - F(\theta)$
    $B \leftarrow B + \frac{(\Delta F - B\Delta\theta)\Delta\theta^T}{\Delta\theta^T\Delta\theta}$
    $\theta \leftarrow \theta_{new}$
  end while
  return θ
end procedure

7.4 Symmetric Rank-1 Update (SR1)

Algorithm 7.4 (Symmetric Rank-1 Update)

The SR1 update is another quasi-Newton method that maintains an approximation H_k of the inverse Hessian, using a symmetric rank-1 correction.

Require: Objective $f(\theta)$, gradient ∇f, initial guess θ_0, tolerance ϵ
procedure SR1($f, \nabla f, \theta_0$)
  $H \leftarrow I$
  while not converged **do**
    $g \leftarrow \nabla f(\theta)$
    $d \leftarrow -Hg$
    $\alpha \leftarrow \text{LineSearch}(f, \theta, d)$
    $\theta_{new} \leftarrow \theta + \alpha d$
    $s \leftarrow \theta_{new} - \theta$
    $y \leftarrow \nabla f(\theta_{new}) - g$
    if $(s - Hy)^T y \neq 0$ **then**
      $H \leftarrow H + \frac{(s - Hy)(s - Hy)^T}{(s - Hy)^T y}$
    end if
    $\theta \leftarrow \theta_{new}$
  end while
  return θ
end procedure

7.5 BFGS

Netwon's method is extremely effective for finding the minimum of a convex function, but there are two disadvantages. First, it is sensitive to initial conditions, and second, it is extremely expensive, with a computational complexity of $O(n^3)$ from having to invert the Hessian. An alternative family of optimizers, called

quasi-Newton methods, try to *approximate* the Hessian (or Jacobian) with $\hat{H}f$, reducing the computational cost without too much loss in convergence rates, and simply use this approximation in the Newton's update:

$$\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - [\hat{H}f_{\mathbf{x}_k}]^{-1}\nabla_{\mathbf{x}} f(\mathbf{x}_k)$$

The method of the Hessian approximation varies by algorithm, but the most popular is BFGS.

So how do we approximate the Hessian with only the gradient information? With secants. Starting off with $f : \mathbb{R} \longrightarrow \mathbb{R}$, let us assume that we have two points $(x_k, f(x_k))$ and $(x_{k+1}, f(x_{k+1}))$. We can approximate our derivative (gradient in dimension 1) at x_{k+1} using finite differences:

$$f'(x_{k+1})(x_{k+1} - x_k) \approx f(x_{k+1}) - f(x_k)$$

and doing the same for f' gives us the second derivative approximation:

$$f''(x_{k+1})(x_{k+1} - x_k) \approx f'(x_{k+1}) - f'(x_k)$$

which gives us the update:

$$x_{k+1} \leftarrow x_k - \frac{x_k - x_{k-1}}{f'(x_k) - f'(x_{k-1})} f'(x_k)$$

This method of approximating Netwon's method in one dimension by replacing the second derivative with its finite difference approximation is called the *secant method*. In multiple dimensions, given two points $\mathbf{x}_k, \mathbf{x}_{k+1}$ with their respective gradients $\nabla f(\mathbf{x}_k), \nabla f(\mathbf{x}_{k+1})$, we can approximate the Hessian $\hat{H}f_{\mathbf{x}_{k+1}} \approx D(\nabla f)_{\mathbf{x}_{k+1}}$ (which is the total derivative of the gradient) at $\mathbf{x}_{k+1}$ with the equation

$$\hat{H}f_{\mathbf{x}_{k+1}}(\mathbf{x}_{k+1} - \mathbf{x}_k) = \nabla_{\mathbf{x}} f(\mathbf{x}_{k+1}) - \nabla_{\mathbf{x}} f(\mathbf{x}_k)$$

This is solving the equation of form $A\mathbf{x} = \mathbf{y}$ for some linear map A. Since $\hat{H}f_{\mathbf{x}_{k+1}}$ is a symmetric $n \times n$ matrix with $n(n+1)/2$ components, there are $n(n+1)/2$ unknowns with only n equations, making this an underdetermined system. Quasi-Newton methods have to impose additional constraints, with the BFGS choosing the one where we want $\hat{H}f_{\mathbf{x}_{k+1}}$ to be as close as to $\hat{H}f_{\mathbf{x}_k}$ at each update $k+1$. Luckily, we can formalize this notion as minimizing the distance between $f_{\mathbf{x}_{k+1}}$ and $\hat{H}f_{\mathbf{x}_k}$. Therefore, we wish to find

$$\arg\min_{\hat{H}f_{\mathbf{x}_{k+1}}} ||\hat{H}f_{\mathbf{x}_{k+1}} - \hat{H}f_{\mathbf{x}_k}||_F,$$

where $||\cdot||_F$ is the Frobenius matrix norm, subject to the restrictions that $\hat{H}f_{\mathbf{x}_{k+1}}$ be positive definite and symmetric and that $\hat{H}f_{\mathbf{x}_{k+1}}(\mathbf{x}_{k+1} - \mathbf{x}_k) = \nabla_{\mathbf{x}} f(\mathbf{x}_{k+1}) - \nabla_{\mathbf{x}} f(\mathbf{x}_k)$ is satisfied. Since we have to invert it eventually, we can actually just create an optimization problem that directly computes the inverse. So, we wish to find

$$\arg\min_{(\hat{H}f_{\mathbf{x}_{k+1}})^{-1}} ||(\hat{H}f_{\mathbf{x}_{k+1}})^{-1} - (\hat{H}f_{\mathbf{x}_k})^{-1}||_F$$

subject to the restrictions that

1. $(\hat{H}f_{\mathbf{x}_{k+1}})^{-1}$ be positive definite and symmetric. It turns out that the positive definiteness restriction also restricts it to be symmetric.
2. $\mathbf{x}_{k+1} - \mathbf{x}_k = (\hat{H}f_{\mathbf{x}_{k+1}})^{-1}[\nabla_{\mathbf{x}} f(\mathbf{x}_{k+1}) - \nabla_{\mathbf{x}} f(\mathbf{x}_k)]$

After some complicated mathematical derivation, which we will not go over here, the problem ends up being equivalent to updating our approximate Hessian at each iteration by adding two symmetric, rank-one matrices U and V scaled by some constant, which can each be computed as an outer product of vectors with itself.

$$\hat{H}f_{\mathbf{x}_{k+1}} = \hat{H}f_{\mathbf{x}_k} + aU + bV = \hat{H}f_{\mathbf{x}_k} + a\mathbf{u}\mathbf{u}^T + b\mathbf{v}\mathbf{v}^T$$

where $\mathbf{u}$ and $\mathbf{v}$ are linearly independent. This addition of a rank-2 sum of matrices $aU + bV$, known as a rank-2 update, guarantees the "closeness" of $\hat{H}f_{\mathbf{x}_{k+1}}$ to $\hat{H}f_{\mathbf{x}_k}$ at each iteration. With this form, we now

impose the quasi-Newton condition. Substituting $\Delta \mathbf{x}_k = \mathbf{x}_{k+1} - \mathbf{x}_k$ and $\mathbf{y}_k = \nabla_{\mathbf{x}} f(\mathbf{x}_{k+1}) - \nabla_{\mathbf{x}} f(\mathbf{x}_k)$, we have

$$\hat{H}f_{\mathbf{x}_{k+1}} \Delta \mathbf{x}_k = \hat{H}f_{\mathbf{x}_{k+1}} \Delta \mathbf{x}_k + a\mathbf{u}\mathbf{u}^T \Delta \mathbf{x}_k + b\mathbf{v}\mathbf{v}^T \Delta \mathbf{x}_k = \mathbf{y}_k$$

A natural choice of vectors turn out to be $\mathbf{u} = \mathbf{y}_k$ and $\mathbf{v} = \hat{H}f_{\mathbf{x}_k} \Delta \mathbf{x}_k$, and substituting this and solving gives us the optimal values

$$a = \frac{1}{\mathbf{y}_k^T \Delta \mathbf{x}_k}, \qquad b = -\frac{1}{\Delta \mathbf{x}_k^T \hat{H}f_{\mathbf{x}_k} \Delta \mathbf{x}_k}$$

and substituting these values back to the Hessian approximation update gives us the BFGS update:

$$\hat{H}f_{\mathbf{x}_{k+1}} = \hat{H}f_{\mathbf{x}_k} + \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \Delta \mathbf{x}_k} - \frac{\hat{H}f_{\mathbf{x}_k} \Delta \mathbf{x}_k \Delta \mathbf{x}_k^T \hat{H}f_{\mathbf{x}_k}}{\Delta \mathbf{x}_k^T \hat{H}f_{\mathbf{x}_k} \Delta \mathbf{x}_k}$$

We still need to invert this, and using the *Woodbury formula*

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1}$$

which tells us how to invert the sum of an intertible matrix A and a rank-k correction, we can derive the iterative update of the inverse Hessian as

$$(\hat{H}f_{\mathbf{x}_{k+1}})^{-1} = \left(I - \frac{\Delta \mathbf{x}_k \mathbf{y}^T}{\mathbf{y}_k^T \Delta \mathbf{x}_k}\right)(\hat{H}f_{\mathbf{x}_k})^{-1}\left(I - \frac{\mathbf{y}_k \Delta \mathbf{x}_k^T}{\mathbf{y}_k^T \Delta \mathbf{x}_k}\right) + \frac{\Delta \mathbf{x}_k \Delta \mathbf{x}_k^T}{\mathbf{y}_k^T \Delta \mathbf{x}_k}$$

Remember that this is the iterative step that we want to actually compute, rather than the ones computing the regular Hessian. The whole point of using the Woodbury formula to derive an inverse update step was to do away with the tedious $O(n^3)$ computations of inverting an $n \times n$ matrix. This rank-2 update also preserves positive-definiteness.

Finally, we can choose the initial inverse Hessian approximation $(\hat{H}f_{\mathbf{x}_{k+1}})^{-1}$ to be the identity I or the true inverse Hessian $(Hf_{\mathbf{x}_{k+1}})^{-1}$ (computed just once), which would lead to more efficient convergence. The pseudocode for BFGS is a bit too long and confusing to include here, but most of the time, we won't be implementing BFGS by hand; efficient and established BFGS optimizers are already in numerous packages. Like most optimizers, BFGS is not guaranteed to converge to the true global minimum.

Algorithm 7.5 (BFGS)

The BFGS algorithm maintains an approximation B_k to the inverse Hessian, updating it using gradient differences and parameter steps. This avoids explicitly computing or inverting the Hessian at each iteration.

Require: Objective $f(\theta)$, gradient ∇f, initial guess θ_0, tolerance ϵ, maximum iterations N_{max}

procedure BFGS($f, \nabla f, \theta_0$)
  $B \leftarrow I$ ▷ Initialize inverse Hessian approximation
  $\theta \leftarrow \theta_0$
  $k \leftarrow 0$
  while $k < N_{max}$ and $||\nabla f(\theta)|| > \epsilon$ **do**
    $g \leftarrow \nabla f(\theta)$
    $d \leftarrow -Bg$
    $\alpha \leftarrow \text{LineSearch}(f, \theta, d)$ ▷ Enforce Wolfe conditions
    $\theta_{new} \leftarrow \theta + \alpha d$
    $s \leftarrow \theta_{new} - \theta$
    $y \leftarrow \nabla f(\theta_{new}) - g$
    if $y^T s \leq 0$ **then**
      break ▷ Curvature condition violated
    end if
    $B \leftarrow \left(I - \frac{sy^T}{y^T s}\right) B \left(I - \frac{ys^T}{y^T s}\right) + \frac{ss^T}{y^T s}$
    $\theta \leftarrow \theta_{new}$
    $k \leftarrow k + 1$
  end while
  return θ
end procedure

Algorithm 7.6 (Limited-memory BFGS)

L-BFGS avoids storing the full inverse Hessian by keeping only the last m update pairs (s_i, y_i). At each step, the search direction is computed using a two-loop recursion. This reduces storage from $O(n^2)$ to $O(mn)$ and is widely used in large-scale optimization.

Require: Objective $f(\theta)$, gradient ∇f, initial θ_0, history size m, tolerance ϵ, maximum iterations N_{max}

procedure L-BFGS($f, \nabla f, \theta_0, m$)
  Initialize $\theta \leftarrow \theta_0$
  Initialize empty history lists S, Y
  $k \leftarrow 0$
  while $k < N_{max}$ and $||\nabla f(\theta)|| > \epsilon$ **do**
    $g \leftarrow \nabla f(\theta)$
    $q \leftarrow g$
    Initialize empty list α ▷ First loop: backward pass
    for $i = |S|$ down to 1 **do**
      $\rho_i \leftarrow 1/(y_i^T s_i)$
      $\alpha_i \leftarrow \rho_i \, s_i^T q$
      $q \leftarrow q - \alpha_i y_i$
    end for
    Choose scalar H_0 (e.g., $H_0 = \frac{s_{k-1}^T y_{k-1}}{y_{k-1}^T y_{k-1}} I$ if available, else I)
    $r \leftarrow H_0 q$ ▷ Second loop: forward pass
    for $i = 1$ to $|S|$ **do**
      $\beta \leftarrow \rho_i \, y_i^T r$
      $r \leftarrow r + s_i(\alpha_i - \beta)$
    end for
    $d \leftarrow -r$ ▷ Search direction
    $\alpha \leftarrow \text{LineSearch}(f, \theta, d)$
    $\theta_{new} \leftarrow \theta + \alpha d$
    $s \leftarrow \theta_{new} - \theta$
    $y \leftarrow \nabla f(\theta_{new}) - g$
    if $y^T s > 0$ **then** ▷ Curvature condition
      Append s, y to S, Y
      if $|S| > m$ **then**
        Remove oldest pair
      end if
    end if
    $\theta \leftarrow \theta_{new}$
    $k \leftarrow k + 1$
  end while
  return θ
end procedure

8 Gradient Free Methods

8.1 Simulated Annealing

Unlike the previous optimizers, *simulated annealing* is useful in finding *global* optima in the presence of multimodal functions within a usually very large discrete space $\mathcal{S}$. Given some function f defined on $\mathcal{S}$, we would like to find its global maximum. Rather than picking the "best move" using gradient information (like SGD), we propose a random move. Let us start at a state θ_k and propose a random P_{k+1}. We denote $\Delta f = f(P_{k+1}) - f(\theta_k)$.

1. If the selected move improves the solution (i.e. $\Delta f \geq 0$, then it is always accepted and we set $\theta_{k+1} \leftarrow P_{k+1}$.
2. Otherwise, when $\Delta f < 0$ it makes the move with the following acceptance probability

$$p(\theta_{k+1} \leftarrow P_{k+1} \mid \Delta f < 0) = e^{\Delta f / T(t)}$$

We can see that if Δf is very negative (the move is very bad), then this probability of acceptance decreases as well. Furthermore, $T(t)$ represents some sort of "temperature" that we anneal as a function of time, called the *annealing schedule*. T starts off at a high value, increasing the rate at which bad moves are accepted, which promotes exploration of $\mathcal{S}$ and allows the algorithm to travel to suboptimal areas. As T decreases, the vast majority of steps move uphill, promoting exploitation, which means that once the algorithm is in the right search space, there is no need to search other sections of the search space.

Algorithm 2 Simulated Annealing

Require: Initial θ_0, Transition kernel $\pi(\theta_{k+1} \mid \theta_k)$, Annealing schedule $T(t)$
  for $t = 0$ to T until convergence **do**
    $P_{t+1} \sim \pi(\cdot \mid \theta_t)$
    if $f(P_{t+1}) - f(\theta_t) \geq 0$ **then**
      $\theta_{t+1} \leftarrow P_{t+1}$
    else
      $\delta \sim \text{Uniform}[0, 1]$
      if $\delta < \exp[(f(P_{t+1}) - f(\theta_t))/T(t)]$ **then**
        $\theta_{t+1} \leftarrow P_{t+1}$
      else
        $\theta_{t+1} \leftarrow \theta_t$
      end if
    end if
  end for

This algorithm is very easy to implement and provides optimal solutions to a wide range of problems (e.g. TSP and nonlinear optimization), but it can take a long time to run if the annealing schedule is very long. We can stop either if T reaches a certain threshold or if we have determined convergence.

8.2 Nelder-Mead

Uses simplex.

9 Lagrangian Optimizers for Constraints

9.1 Lagrange Multipliers

For equality.

9.2 KKT Conditions

For inequality.

9.3 ADMM

10 Non-Lagrangian Optimizers for Constraints

10.1 Penalty

10.2 Projection

10.3 Saddle Point Problem in Nonconvex Optimization

[PDGB14]

11 Sparsity-Inducing Optimizers

11.1 Clipping

We can do SGD with clipping.

References

[PDGB14] Razvan Pascanu, Yann N. Dauphin, Surya Ganguli, and Yoshua Bengio. On the saddle point problem for non-convex optimization, 2014.